

Introduction

What are computers?

Talking about general purpose, binary, digital computers

Dumb machines that need to be told what to do

How do we tell them what to do?

By giving them instructions

How do we communicate with them?

Use special languages such as Assembly, FORTRAN, C, Python etc.

What is a program?

Precise set of instructions to solve a problems

Art of writing these instructions is called programming

Programming Languages

How are instructions written?

Human readable form

What are programming languages?

Specific languages developed to write computer instructions

Some are machine like languages such as assembly or low level languages

Others are human readable or high level languages such as FORTRAN, COBOL, C, C++, Python

Compiled vs interpreted languages

How do we transform a human readable program into machine understandable

Compilation vs Interpretation

Example: Need to cook meat to eat – compiling, Salad vegetables can be eaten as is - interpreting

Language for the course: Python

Why Python is popular

Easy to learn: straightforward syntax and very readable

Versatile: can be used for a wide variety of applications, including web development, data analysis, and scientific computing.

Automation: powerful tool for automating repetitive tasks, such as file handling, web scraping, and software testing.

Machine learning and AI support: popular choice for machine learning, artificial intelligence, and data science.

Active community: Python has a large and supportive development community.

Vast ecosystem: Python has a large ecosystem of libraries for data processing, machine learning, data science, and more.

Setting up the Python environment on your computer

Go to the official Python website (<https://www.python.org/>)
Select the latest version of Python
Select the "Windows installer" option
Click "Download"
Run the installer
Follow the on-screen instructions
Verify the installation by opening a command prompt and typing `python -V` (or `py -V`)
Run Python and display some interactive commands and results

Setting up VS Code as the preferred editor

Go to the official Visual Studio Code site: <https://code.visualstudio.com/>
Locate the latest version of VS Code for your operating system
Download the installer as an .exe file.
Install the downloaded file:
Run the installer and click Next until you reach Finish.
Choose support for Python when prompted

Our first Python program

Open VS Code
Type the code
`Print("Hello World")`
Save as a .py file
Run the program highlighting the output
Using the Python CLI when needed (will be used in operators module etc.)

Programming Basics

To design a program properly
Analyze the problem
Express its essence, with examples
Formulate statements precisely
Evaluate, test and revise if necessary
Pay attention to detail

Example program:

Find the weight of metal required to make an annular ring with inner and outer radii 2 cm and 5 cm. The metal sheet has a grammage of 10 gms/cm². Assume $\pi=3.14$

Walk through analysis, variables, precision, testing etc. (with diagrams)

r=2cm

r=5cm

The program

Find area of larger circle:

$A1 = 3.14 * 5 * 5$

Find area of smaller circle:

$A2 = 3.14 * 2 * 2$

Find area of metal sheet to make the ring

$A3 = A1 - A2$

Find the weight of the ring:

$W = A3 * 10$

Answer W is in gms

Writing this program in Python (screen rec.)

Find area of larger circle:

$A1 = 3.14 * 5 * 5$

Find area of smaller circle:

$A2 = 3.14 * 2 * 2$

Find area of metal sheet to make the ring

$A3 = A1 - A2$

Find the weight of the ring:

$W = A3 * 10$

Answer W is in gms

print("The weight of the metal ring is: ", W, " grams.")

Python Code: ring.py

```
# Find area of larger circle:
A1 = 3.14 * 5 * 5
# Find area of smaller circle:
A2 = 3.14 * 2 * 2
# Find area of metal sheet to make the ring
A3 = A1 - A2
# Find the weight of the ring:
W = A3 * 10
# Answer W is in gms
print("The weight of the metal ring is: ", W, " grams.")
```

Python Code: 1_case_sensitive.py

```
#
# Case sensitivity
#
# Find area of larger circle:
```

```

A1 = 3.14 * 5 * 5
# Find area of smaller circle:
A2 = 3.14 * 2 * 2
# Find area of metal sheet to make the ring
A3 = A1 - A2
# Find the weight of the ring:
W = A3 * 10
# Answer W is in gms
print("The weight of the remaining metal ring is: ", W, " grams.")

```

Python Code: 2_colon_Indent.py

```

#
#   Statement blocks - colon and indentation
#
# Find area of larger circle:
A1 = 3.14 * 5 * 5          # This is an inline comment
# Find area of smaller circle:
A2 = 3.14 * 2 * 2
# Find area of metal sheet to make the ring
A3 = A1 - A2
# Find the weight of the ring:
W = A3 * 10
# Answer W is in gms
if(W>100):
    print("The area of the outer disc is ",A1)
    print("The area of the inner disc is ",A2)
print("The weight of the remaining metal ring is: ", W, " grams.")

```

Python Syntax

Set of rules that define how to write Python programs

Readable: Uses English keywords and indentation

No punctuation

Case sensitive

Reduces errors (missing braces etc.)

Focuses on logic rather than syntax rules

Code Blocks and Comments

Uses indentation to define code blocks

A colon (:) starts a block

All lines in a block should have the same indentation

Use spaces or a tab as indentation

Any number of spaces, but consistently in a single block

Comments

Comments are indicated by a sharp sign (#)

Stand-alone or inline comments are both allowed

The end of a statement is marked by the end of the line

Python Code: 3_input_fn.py

```
#
#   The input function returns a string
#
name=input("Please enter your name: ")
print("Hello", name, type(name))
n = input("Please enter a number:")
print(n, type(n))
x=n+10
```

Reserved Words

Basic Communications

Print function

Used for output

Overloaded for different datatypes so it knows how to handle them
(more on print statement later)

Input function

Returns a string (always)

Can be used to prompt for user input

Python Code: 4_basic_variable_types.py

```
#
#   Some basic variable types
#
a="Hello World"      # str
b=10                 # int
c=10.23              # float
d=complex(1,2)       # complex

print(a, type(a))
print(b, type(b))
print(c, type(c))
print(d, type(d))
```

Variables

What are variables

Variables created when value assigned; no declaration

Variables can change types through a new assignment

The type of a variable can be forced by casting the RHS

The simple assignment operator (=)

Variable Naming Conventions

A variable name must start with a letter or the underscore character

A variable name cannot start with a number

A variable name can only contain alpha-numeric characters and underscores

(A-z, 0-9, and _)

Variable names are case-sensitive (age, Age and AGE are three different variables)

A variable name cannot be any of the Python keywords.

Many values can be assigned to multiple variables: (bad idea)

```
x, y, z = 10, "Banana", "Cherry"
```

Variable Data Types

str

int, float, complex

list, tuple, dict

set, frozenset

bool

bytes, bytearray, memoryview

NoneType

Python Code: 5_arithmetic_operators.py

```
#
#   Arithmetic Operators
#
# # Addition      +
# a=10
# b=20
# c=a+b
# print("The sum of ",a, " and ", b , " is: ", c)
# # Subtraction  -
# a=100
# b=20
# c=a-b
# print("The difference of ",a, " and ", b , " is: ", c)
# # Multiplication *
# a=10
# b=20
# c=a*b
# print("The product of ",a, " and ", b , " is: ", c)
# # Division      /
# a=10
# b=20
# c=a/b
# print(a, " divided by ", b, " is: ", c)
# # Floor Division //
# a=101
# b=20
# c=a//b
# print("The result of floor division of ",a," by ",b, " is: ", c)
```

```

# # Modulus          %
# a=10
# b=3
# c=a%b
# print("The remainder of ",a," divided by ",b, " is: ", c)
# # Exponentiation  **
a=10
b=20
c=a**b
print("The result of ",a," raised to the power ",b, " is: ", c)

```

Operators

Arithmetic operators

Assignment operators

Comparison operators

Logical operators

Identity operators

Membership operators

Bitwise operators

Arithmetic operators

Addition +

Subtraction -

Multiplication *

Division /

Floor Division //

Modulus %

Exponentiation **

Python Code: 6_assignment_operators.py

```

#
# Assignment operators
#
a = 10
# # Simple assignment operator =
print("Simple assignment, a =",a)
# Increment operator +=
a = a + 5
print("Adding 5 to a gives: ",a)
a += 5
print("Incrementing a by 5 gives: ",a)
# # -=

```

```

# a = a-7
# print("Subtracting 7 from a gives: ",a)
# a -= 7
# print("Decrementing a by 7 gives: ",a)
# *=
# a = a*5
# print("Multiplying a by 5 gives: ",a)
# a *= 5
# print("Multiplying a by 5 gives: ",a)
# # /=
# a = a/3
# print("Dividing a by 3 gives: ",a)
# a /= 3
# print("Dividing a by 3 gives: ",a)
# # //=
# a = a//3
# print("Floor division of a by 3 gives: ",a)
# a //= 3
# print("Floor division of a by 3 gives: ",a)
# # %=
# a = a%4
# print("a modulo 4 gives: ",a)
# a %= 4
# print("a modulo 4 gives: ",a)
# **=
# a = a**8
# print("a raised to the power 8: ",a)
# a **= 8
# print("a raised to the power 8: ",a)

```

Assignment operators

```

=
+=
-=
*=
/=
//=
%=
**=

```

Expressions

A combination of operators and operands that is interpreted to produce another value

E.g.

`c = a+b`

Python Code: 7_comparison_operators.py

```
#
#   Comparison operators
#
a = 10
b = 10
# # Equals: a == b
# print("a = ",a, "b = ",b, ", a == b:", a==b)
# # Not Equals: a != b
print("a = ",a, "b = ",b, ", a != b:", a!=b)
# # Less than: a < b
# print("a = ",a, "b = ",b, ", a < b:", a<b)
# # Less than or equal to: a <= b
# print("a = ",a, "b = ",b, ", a <= b:", a<=b)
# # Greater than: a > b
# print("a = ",a, "b = ",b, ", a > b:", a>b)
# # Greater than or equal to: a >= b
# print("a = ",a, "b = ",b, ", a >= b:", a>=b)
```

Comparison operators

Equals: a == b

Not Equals: a != b

Less than: a < b

Less than or equal to: a <= b

Greater than: a > b

Greater than or equal to: a >= b

Python Code: 8_if_statement.py

```
#
#   The if statement
#
a = 10
b = 10
if (a>b):
    print(a, "is greater than", b)
# elif (a == b):
#     print(a,"is equal to",b)
# else:
#     print(a,"is less than",b)
```

The if Statement

Syntax:

if (condition):

 statement(s)

elif (some other condition):

 statements(s)

else:
 statements

Python Code: 9_quadratic_equation.py

```
#
#   Solving a quadratic equation
#
#   Obtain the coefficients of the quadratic equation from the user
#
a=float(input("Enter the coefficient of x**2:"))
b=float(input("Enter the coefficient of x:"))
c=float(input("Enter the constant term:"))
#
#   Calculate the discriminant
#
disc=b*b - 4.0*a*c
#
#   And now simply apply the formula for the roots of a quadratic equation
#
x1=(-b + disc**0.5)/(2*a)
x2=(-b - disc**0.5)/(2*a)
print("The roots of the equation ",a,"x**2 +",b,"x +",c, "= 0 are:")
print("x1 = ",x1, "x2 = ",x2)
```

Putting a few concepts together

Solving a quadratic equation:

$$ax^2 + bx + c = 0 \quad a \neq 0$$

Roots are

$$x = [-b \pm \sqrt{b^2 - 4ac}] / 2a$$

Python Code: 1_and.py

```
#
#   And truth table
#
print("And truth table")
print("a \tb\t a and b")
a = False
b = False
print(a,"\\t",b,"\\t",a and b)
#   print("a = ",a,"b = ",b, ", result of (a and b) = ",a and b)
b = True
#   print("a = ",a,"b = ",b, ", result of (a and b) = ",a and b)
print(a,"\\t",b,"\\t",a and b)
a = True
b = False
print(a,"\\t",b,"\\t",a and b)
#   print("a = ",a,"b = ",b, ", result of (a and b) = ",a and b)
```

```

a = True
b = True
print(a, "\t", b, "\t", a and b)
print("a = ", a, "b = ", b, " ", result of (a and b) = ", a and b)
#
#   Or truth table
#
print("Or truth table")
a = False
b = False
print("a = ", a, "b = ", b, " ", result of (a or b) = ", a or b)
b = True
print("a = ", a, "b = ", b, " ", result of (a or b) = ", a or b)
a = True
b = False
print("a = ", a, "b = ", b, " ", result of (a or b) = ", a or b)
a = True
b = True
print("a = ", a, "b = ", b, " ", result of (a or b) = ", a or b)
#
#   Not truth table
#
a = True
b = not a
print("a = ", a, "b = not a = ", b)

```

Python Code: 2_or.py

```

a = False
b = False
print("a = ", a, "b = ", b, " ", result of (a or b) = ", a or b)
b = True
print("a = ", a, "b = ", b, " ", result of (a or b) = ", a or b)
a = True
b = False
print("a = ", a, "b = ", b, " ", result of (a or b) = ", a or b)
a = True
b = True
print("a = ", a, "b = ", b, " ", result of (a or b) = ", a or b)

```

Logical Operators

and

or

not

Usually combined with comparison operators

and all expressions evaluate to True, then result is True

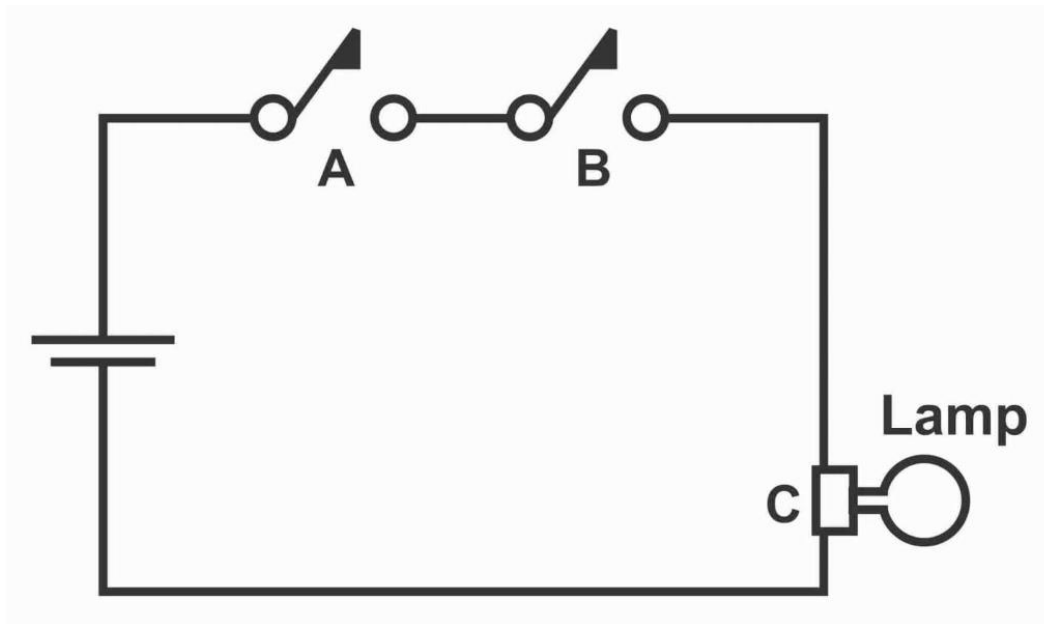
or any expression evaluates to True then result is True
not inverts the value of a logical expression

and Truth Table

$C = A \text{ and } B$

1 = True

0 = False

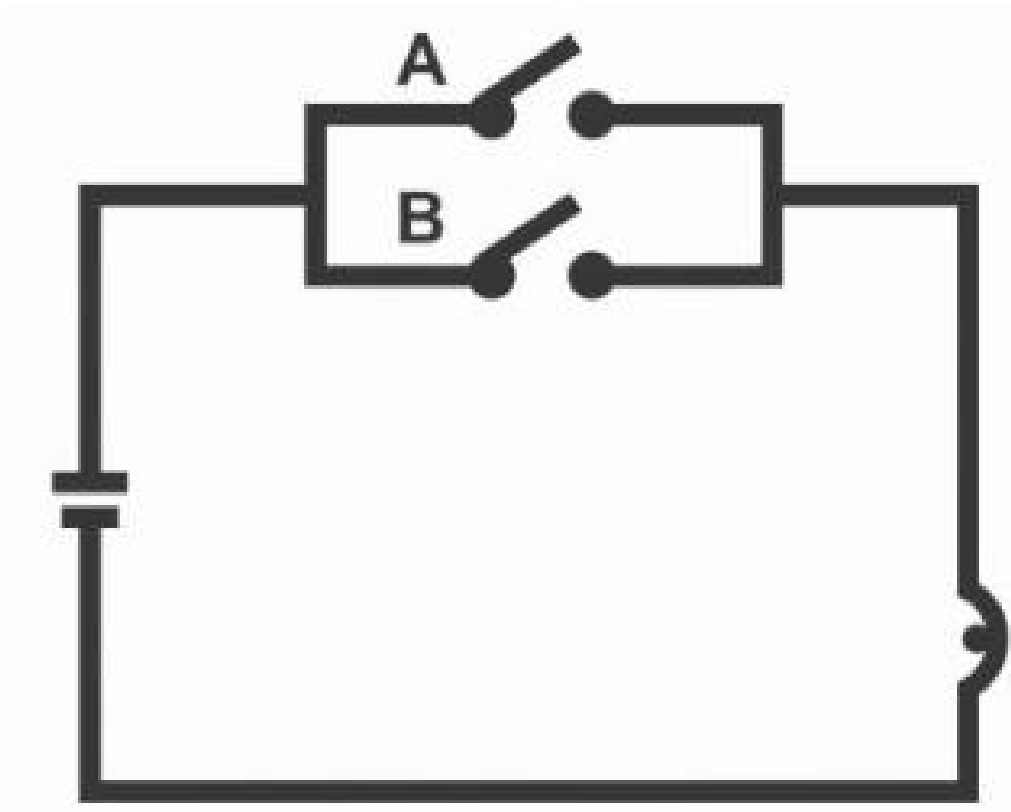


or Truth Table

$C = A \text{ or } B$

1 = True

0 = False



C

not Truth Table

C = not A

1 = True

0 = False

Python Code: 3_age.py

```
25#
#   Example of multiway decision without elif
#
#   May also be written as nested if statements without logical operators
#
age=int(input("Please enter your age:"))
# print("\nUsing sequential if statements with logical operators")
# if (age < 13):
#     print("You are a child")
# if(age >=13 and age <= 19):
#     print("You are a teen")
# if(age >= 20 and age <= 59):
#     print("You are an adult")
# if(age >= 60):
#     print("You are a senior")
print("\nUsing nested if statements:")
if (age < 13):
```

```
        print("You are a child")
else:
    if(age <= 19):
        print("You are a teen")
    else:
        if(age <= 59):
            print("You are an adult")
        else:
            print("You are a senior")
```

Example using logical operators

Write a Python program that takes a user's age as input and then categorizes the person as
A 'Child' (age< 13),
A 'Teen' (13-19),
An 'Adult' (20-59),
A 'Senior' (60 and above)
using only if-else statements (no elif allowed).

Flow Charts

A technique to analyze problems without writing code

We will use a technique called "Structured Flow Charts"

Originally devised by DEC engineer

Use very few symbols and indent only left to right

Start / Stop symbol

Process symbol

Decision symbol

Continuation symbol

Flow Charts – Simple sequential problem

Start

Process

Stop

If statement

True

Process A

Process B

Condition?

If

False

Indentation level

Nested If

Indentation Level 1

Indentation Level 2

Python Code: 5_while_1.py

```
#
#   Example of while loop using doctor's code!
#
#   Also show examples of break, continue, else
i=100
while(i > 0):
    print(i)
    i -= 7
    if(i<20):
        break
    print("Still inside the loop")
else:
    print("The loop has ended")
```

Loops

Used whenever we need to repeat a task

Two broad varieties

“while” loops – run while some condition is true

“for” loops – run for a specific number of times

While loop

Syntax:

```
while(some condition is true):  
    Perform some instructions
```

```
optional:    break  
            continue  
            else
```

Loop runs as long as condition is true

Must be careful to avoid infinite loops

while

Exit

Python Code: 4_range.py

```
#  
# The range function and membership operator  
#  
x = range(10)  
print(x, type(x))  
guess = int(input("Please enter a number:"))  
# Membership operator  
if (guess in x):  
    print("The membership operator returns", guess in x)  
    print("The number entered is in range")  
else:  
    print("The number is NOT in the range")
```

The range function

Syntax:

```
range(number)  
range(start, stop)  
range(start, stop, step)  
where all arguments are integers
```

Returns a “range” object that can be used to iterate through
Most useful in for-loops

The membership operator

Syntax:

```
i in range(number)
(range could be any other group like list etc.)
```

Returns True if i is in the range

Returns False otherwise

Useful in for-loops

For loop

Used for iterating over a sequence

E.g. a range, a list, a tuple, a dictionary, a set, a string

Syntax:

```
for x in <some sequence>:
    Perform some instructions
```

```
optional:    break
              continue
              else
```

Loop runs through all values in the sequence (zero or more)

Process & Next var

Var in sequence?

for

Init loop var

Optional loop statements

continue

forces next iteration of the loop skipping any remaining statements

else

executed after normal termination of the loop

break

exits loop immediately not executing any “else” statements

Python Code: 6_for.py

```
#
#   Example of for loop using range function and simple counting
#
#   Also show examples of break, continue, else
#   Even, odd, sum of etc.
#
for i in range(1,10,2):      # options: range(1,10), range(1,10,2)
    print (i)
else:
    print("THE LOOP EXITED SUCCESFULLY")
```

Used for iterating over a sequence

E.g. a range, a list, a tuple, a dictionary, a set, a string

Syntax:

```
for x in <some sequence>:
    Perform some instructions
```

```
optional:      break
               continue
               else
```

Loop runs through all values in the sequence (zero or more)

continue

forces next iteration of the loop skipping any remaining statements

else

executed after normal termination of the loop

break

exits loop immediately not executing any “else” statements

Python Code: 1_sum_n.py

```
#
#   Sum of first n natural numbers
#
print("Sum of the first n natural numbers where n is entered by the user:\n")
n=int(input("Enter a positive integer:"))
sum = 0
for i in range(1,n+1):
    sum += i
print("\nThe sum of the first",n,"natural numbers is:",sum,"\n")
```

Some simple loop examples

Manual computation vs loops

Counting and Sums

Sum of first n integers

How many numbers between a and b are exactly divisible by c?
(Using debugger to observe execution)

What is the sum of integers between a and b inclusive?

What is the sum of even numbers between a and b inclusive?

Python Code: 2_count_divisible.py

```
#
# Finding how many numbers between two limits are exactly divisible by a given
number
#
a = int(input("Enter starting number:"))
b = int(input("Enter the ending number:"))
c = int(input("Enter the number to divide by:"))
count = 0
for index in range(a, b+1):
    if (index%c == 0):
        count += 1
print("There are",count,"numbers between",a,"and",b,"that are exactly divisble
by",c)
```

Python Code: 3_sum_a_to_b.py

```
#
# Find the sum of integers between a and b inclusive
#
a = int(input("Enter starting number:"))
b = int(input("Enter the ending number:"))
sum = 0
for index in range(a, b+1):
    sum += index
print("The sum of all integers between",a,"and",b,"is",sum)
```

Python Code: 4_sum_even_a_to_b.py

```
#
# Find the sum of even integers between a and b
#
a = int(input("Enter starting number:"))
b = int(input("Enter the ending number:"))
sum = 0
for index in range(a, b+1):
    if index%2 == 0:
        sum += index
print("The sum of all even integers between",a,"and",b,"is",sum)
```

Python Code: 5_power.py

```
#
# x ** n using a loop
```

```
#
print("Calculate base number raised to a specified power using a loop:\n")
x=float(input("Enter the base number:"))
n=int(input("Enter the power to which it should be raised (0 or higher):"))
norig = n
result=1
while(n>0):
    result *= x
    n -= 1
print("\n",x,"raised to the power",norig,"is",result,"\n")
print("\nConfirmed using built-in x**n formula:\n",x,"raised to the
power",norig,"is",x**norig,"\n")
```

Python Code: 6_factorial_loop.py

```
# Calculate the factorial of a number using a loop
#
n=int(input("Enter a positive integer:"))
fac=1
for i in range(1,n+1):
    fac *= i
print("The factorial of",n,"is", fac)
```

Python Code: 7_1_fn_simple.py

```
#
# Simple function without arguments and no return value
#
def fineprint():
    print("This is standard text printed at the bottom\n")

for i in range(5):
    print("This is the contract for person",i+1)
    fineprint()
```

Functions

Code blocks used to execute a certain defined process

Used to break long complex code into easily understood units

Most useful when the same code block is to be used repeatedly

Functions – Contd.

Defined using the “def” keyword

Parentheses must follow the name of the function, with or without arguments

Called by giving the name of the function followed by parentheses, again, with or without arguments

Can accept arguments (one or more) and return result(s)

Functions only run when called

Use a “return” statement to return value(s)

Number of arguments used to call must match the number of parameters defined in the function and be in the correct order

Alternately, use key=value syntax in calling args, then order does not matter. However, key MUST be defined in the function

A parameter may be given a default value in the function's def statement. This value is used if none provided in the calling statement. All params following the one with default value, MUST have default values defined

Any data type may be passed to a function and it will be treated as the same type inside the function (e.g. List, etc.)

Multiple values can be returned as a tuple, a list, a dictionary, a set etc.

If function uses * before parameter, then it may be called with a variable number of args and params will be accessed by indexing

Function then processes the params as received

If number of key values is not known, use ** before parameter and function will decipher accordingly by using a dictionary method

Recursion

When a function calls itself directly or indirectly

Used when a task can be divided into identical sub-tasks

Basic structure of a recursive function:

```
def recursive_function(parameters):  
    if base_case_condition:  
        return base_result  
    else:  
        return recursive_function(modified_parameters)
```

Examples:

Sum by recursion

Factorial by recursion

Fibonacci series by recursion

Python Code: 7_2_fn_args_retval.py

```
#  
#   Function with arguments and a return value  
#  
def compute(a,b,c):  
    print (type(a),type(b),type(c))  
    return (a+b+c)  
  
print(type(compute(2,3,40.5)))
```

Number of arguments used to call must match the number of parameters defined in the function and be in the correct order

Alternately, use key=value syntax in calling args, then order does not matter. However, key MUST be defined in the function

A parameter may be given a default value in the function's def statement. This value is used if none provided in the calling statement. All parms following the one with default value, MUST have default values defined

Any data type may be passed to a function and it will be treated as the same type inside the function (e.g. List, etc.)

Multiple values can be returned as a tuple, a list, a dictionary, a set etc.

Python Code: 7_2_fn_payroll.py

```
#
#   Function to compute payroll
#
def payroll(person,basic):
    house_rent=0.45 * basic
    utilties = 0.10 * basic
    gross_pay = basic + house_rent + utilties
    tax = 0.15 * gross_pay
    net_pay = gross_pay - tax
    print(person,"\tGross Salary = ",gross_pay,"\tTax = ", tax, "\tNet Salary = ",net_pay)

for i in range(1,5):
    name = "Person"+str(i)
    basic_salary = i * 30000
    payroll(name, basic_salary)
```

Python Code: 7_3_fn_args_tuple.py

```
#
#   Function with arguments and multiple return values (Special to Python)
#
def compute(a,b,c):
    return 2*a,3*b,4*c

t = compute(2,3,40) # returns a tuple
print(t, type(t))
#
# Unpack a tuple
#
x,y,z=t
print("x=",x, "y=",y, "z=",z)
```

Python Code: 7_4_fn_key_val.py

```
#
#   Function with key=value syntax, position no longer relevant
#
def compute(a,b,c):
    return 2*a,3*b,4*c

print(compute(b=2,a=3,c=40)) # returns a tuple
```

Number of arguments used to call must match the number of parameters defined in the function and be in the correct order

Alternately, use key=value syntax in calling args, then order does not matter. However, key MUST be defined in the function

A parameter may be given a default value in the function's def statement. This value is used if none provided in the calling statement. All params following the one with default value, MUST have default values defined

Any data type may be passed to a function and it will be treated as the same type inside the function (e.g. List, etc.)

Multiple values can be returned as a tuple, a list, a dictionary, a set etc.

Python Code: 7_5_fn_default_val.py

```
#
#   Function with key=value syntax and default values
#
def compute(a,b=15,c=88):
    return 2*a,3*b,4*c

print(compute(a=1,b=2,c=3)) # returns a tuple
```

When a function calls itself directly or indirectly

Used when a task can be divided into identical sub-tasks

Basic structure of a recursive function:

```
def recursive_function(parameters):
    if base_case_condition:
        return base_result
    else:
        return recursive_function(modified_parameters)
```

Python Code: 91_sum_recursion.py

```
def sum_rekurs(k):
    #
    # Base case
    #
    if(k==0):
        return 0
    #
    # Recursive case
    #
    else:
        return k + sum_rekurs(k-1)

#
#   Sum of first n natural numbers using recursion
#
print("Sum of the first n natural numbers where n is entered by the user:\n")
n=int(input("Enter a positive integer:"))
```

```
print("The sum of the first",n,"natural numbers is:",sum_rekurs(n))
```

Python Code: 92_factorial_recursion.py

```
#
#   Factorial of a number using recursion
#
def factorial(n):
    if (n == 1):
        return(1)
    else:
        return(n*factorial(n-1))

n=int(input("Enter a positive integer:"))
print("The factorial of",n,"is",factorial(n))
```

Python Code: 93_fibonacci.py

```
def fib(n):
    if (n==0):
        return(0)
    elif (n==1):
        return(1)
    else:
        return(fib(n-1) + fib(n-2))

n=int(input("Enter a positive integer:"))
print("The ",n,"th Fibonacci number is:", fib(n))
```

Python Code: 0_strings (1).py

```
print("This is a string")
#
#   Define str1 as a string variable
#
str1 = "This is a new string"
print(str1)
#
#   A string with enclosed quotes
#
str2 = "This is a string with 'enclosed' quotes"
print(str2)
str3 = 'This is a string with "enclosed" quotes'
print(str3)
str4="""This is a multiline string. The linebreaks
are preserved as is and reflected in the
output"""
print(str4)
```

Python Code: 0_strings.py

```
print("This is a string")
#
#   Define str1 as a string variable
```

```
#
str1 = "This is a new string"
print(str1)
#
#   A string with enclosed quotes
#
str2 = "This is a string with 'enclosed' quotes"
print(str2)
str3 = 'This is a string with "enclosed" quotes'
print(str3)
str4="""This is a multiline string. The linebreaks
are preserved as is and reflected in the
output"""
print(str4)
```

Strings

Defined using single or double quotes

We can put any character(s) within the quotes

If defined using “ we can include ‘ within the string and vice versa

String variables are strings assigned to a variable

Multi-line strings are enclosed in triple quotes

Strings are immutable

We use the print() function to display a string literal or variable

Defined using single or double quotes

We can put any character(s) within the quotes

If defined using “ we can include ‘ within the string and vice versa

String variables are strings assigned to a variable

Multi-line strings are enclosed in triple quotes

Strings are immutable

We use the print() function to display a string literal or variable

Python Code: 1_str_py_fns.py

```
#
#   Python functions acting on strings
#   The type(), len() and str() functions
#
str1 = "This is a test"
print(str1, type(str1), len(str1))
#
#   Casting into a string
#
x = 123.456
strx = str(x)
print(x, type(x), strx, type(strx))
```

Python functions applicable to strings

`type()` applicable to all Python data types, including strings

`str()` create a string object

`len()` returns the length of a string

Python Code: 2_str_arrays (1).py

```
# #
# #   A string can be treated as an array with indexing
# #
str1 = "This is a test"
# strlen = len(str1)
# print(str1, strlen)

# for i in range(strlen): # Using indexing
#     print(str1[i])

# for c in str1:          # Using membership operator (cleaner)
#     #
#     #   ignore all vowels and white space
#     #
#     if c not in "aeiou ":
#         print(c)
# #
# #   Extracting a substring
# #
# substr = str1[1:5]
# print(substr)
# #
# #   However, strings are immutable and the following will result in an error
# #
str2 = str1[0:4]+'Z'+str1[5:len(str1)]
print(str2)
```

Python Code: 2_str_arrays.py

```
# #
# #   A string can be treated as an array with indexing
# #
str1 = "This is a test"
# strlen = len(str1)
# print(str1, strlen)

# for i in range(strlen): # Using indexing
#     print(str1[i])

# for c in str1:          # Using membership operator (cleaner)
#     #
#     #   ignore all vowels and white space
#     #
#     if c not in "aeiou ":
#         print(c)
# #
# #   Extracting a substring
```

```
# #
# substr = str1[1:5]
# print(substr)
# #
# #     However, strings are immutable and the following will result in an error
#
str2 = str1[0:4]+'Z'+str1[5:len(str1)]
print(str2)
```

Strings as arrays

Strings are arrays of characters

A single character is a string of length 1

We can index through a string using [] starting at base 0

We can extract a substring using the str[start:end] technique

Since strings are arrays, we can loop through a string using a for loop and a membership operator

Strings are immutable

Strings are arrays of characters

A single character is a string of length 1

We can index through a string using [] starting at base 0

We can extract a substring using the str[start:end] technique

Since strings are arrays, we can loop through a string using a for loop and a membership operator

Strings are immutable

Python Code: 3_str_escape.py

```
#
#     Escape characters
#     Use a \ to print special characters
#
str1 = "This is a string with \n a newline embedded"
print(str1)
str2 = "This is a string with \t a tab embedded"
print(str2)
str3 = "A string with a carriage \r return embedded"
print(str3)
str4 = "A string with a formfeed \f embedded"
print(str4)

str5 = "This is a string in double quotes with a \"quoted\" word embedded inside"
print(str5)
str6 = 'Of course we can do the \'same\' with single quotes!'
print(str6)

str7 = "What if we need to embed a \"\\\"?"
print(str7)
str8 = "A string with an octal value: \156"
print(str8)
```

```
str9 = "A string with a hex value: \x6E"  
print(str9)
```

Escape sequences

Used when we need a special character to be part of a string

<code>\n</code>	<code>\t</code>	<code>\r</code>	<code>\f</code>
<code>\'</code>	<code>\"</code>		
<code>\\</code>			
<code>\nnn</code>	octal value		
<code>\xnn</code>	hexadecimal value		

Python Code: 4_str_upper_lower (1).py

```
# capitalize()      first character of string to uppercase, force rest to lower  
tststr = "the quick BROWN fox Jumped over the lazy dog"  
str1 = tststr.capitalize()  
print(str1)  
# casefold()  converts string to lowercase (ascii & non-ascii)  
str2=tststr.casefold()  
print(str2)  
# lower()      converts string to lowercase  
str3=tststr.lower()  
print(str3)  
# title()      converts first character of each word to upper case  
str4=tststr.title()  
print(str4)  
# upper()      converts string to uppercase  
str5=tststr.upper()  
print(str5)  
# swapcase()  lower case becomes upper case and vice versa  
str6=tststr.swapcase()  
print(str6)
```

Python Code: 4_str_upper_lower.py

```
# capitalize()      first character of string to uppercase, force rest to lower  
tststr = "the quick BROWN fox Jumped over the lazy dog"  
str1 = tststr.capitalize()  
print(str1)  
# casefold()  converts string to lowercase (ascii & non-ascii)  
str2=tststr.casefold()  
print(str2)  
# lower()      converts string to lowercase  
str3=tststr.lower()  
print(str3)  
# title()      converts first character of each word to upper case  
str4=tststr.title()  
print(str4)  
# upper()      converts string to uppercase
```

```
str5=tststr.upper()
print(str5)
# swapcase() lower case becomes upper case and vice versa
str6=tststr.swapcase()
print(str6)
```

String methods (functions)

Conversion functions:

Syntax: string.function() returns a new string

capitalize() first character of string to uppercase, rest to lower

casefold() converts string to lowercase (ascii & non-ascii)

lower() converts string to lowercase

title() converts first character of each word to upper case

upper() converts string to uppercase

swapcase() lower case becomes upper case and vice versa

Conversion functions:

Syntax: string.function() returns a new string

capitalize() first character of string to uppercase, rest to lower

casefold() converts string to lowercase (ascii & non-ascii)

lower() converts string to lowercase

title() converts first character of each word to upper case

upper() converts string to uppercase

swapcase() lower case becomes upper case and vice versa

Python Code: 5_str_strip.py

```
# Stripping white space or other characters
tststr=" The quick brown fox "
print(tststr, len(tststr))
str1=tststr.lstrip()
print(str1, len(str1))
str2=tststr.rstrip()
print(str2, len(str2))
str3=tststr.strip()
print(str3, len(str3))
str4=tststr.strip(" This")
print(str4, len(str4))
str5=tststr.strip("x ")
print(str5, len(str5))
```

String methods (contd.)

Trimming leading and trailing white space:

`lstrip()`
`rstrip()`
`strip()`

Optional argument: Set of characters in quotes that will be stripped (default: space)

Python Code: 5a_str_just.py

```
# Justifying strings
tststr="The quick brown fox"
print(tststr.ljust(50))
print(tststr.rjust(50))
print(tststr.center(50))
print(tststr.ljust(50, '='))
print(tststr.rjust(50, '='))
print(tststr.center(50, '='))
#
# e.g. amounts on cheques are right justified with preceding fill characters
#
print("100,234.56".rjust(20, '*'))
```

Justification:

<code>ljust()</code>	left justified
<code>rjust()</code>	right justified
<code>center()</code>	centered string

All require at least one argument (the total width of the string) and an optional fill character

Left justified

Right Justified

Centered

width

Python Code: 6_str_count (1).py

```
#
# Counting the number of times a substring is found in a string
#
tststr="    The quick brown fox    "
for c in " abcdefghijklmnopqrstuvwxyz":
    print(c, tststr.count(c))
```

```

    print(c.upper(), tststr.count(c.upper()))
print("qu",tststr.count("qu"))

```

Python Code: 6_str_count.py

```

#
#   Counting the number of times a substring is found in a string
#
tststr="    The quick brown fox    "
for c in " abcdefghijklmnopqrstuvwxyz":
    print(c, tststr.count(c))
    print(c.upper(), tststr.count(c.upper()))
print("qu",tststr.count("qu"))

```

count() Returns number of times a string occurs in calling string

Arg: string containing one or more characters (required).

Empty string => all characters are counted

count() Returns number of times a string occurs in calling string

Arg: string containing one or more characters (required).

Empty string => all characters are counted

Python Code: 7_str_find (1).py

```

#
#   Find the location of a substring in a string
#
tststr="    The quick brown fox    "
lookfor = "o"
n = tststr.find(lookfor,0,10)
#
#   Returns -1 if not found
#
if n != -1:
    print(lookfor,"occurs at position",n)
else:
    print(lookfor,"is not in the string")
#
#   index() does the same thing but throws an exception if not found
#
n = tststr.index(lookfor)
print(n)

n = tststr.rfind(lookfor)
#
#   Returns -1 if not found
#
if n != -1:
    print(lookfor,"occurs at position",n)
else:
    print(lookfor,"is not in the string")
#
#   index() does the same thing but throws an exception if not found

```

```
#
n = tststr.rindex(lookfor)
print(n)
```

Python Code: 7_str_find.py

```
#
# Find the location of a substring in a string
#
tststr="    The quick brown fox    "
lookfor = "o"
n = tststr.find(lookfor,0,10)
#
# Returns -1 if not found
#
if n != -1:
    print(lookfor,"occurs at position",n)
else:
    print(lookfor,"is not in the string")
#
# index() does the same thing but throws an exception if not found
#
n = tststr.index(lookfor)
print(n)

n = tststr.rfind(lookfor)
#
# Returns -1 if not found
#
if n != -1:
    print(lookfor,"occurs at position",n)
else:
    print(lookfor,"is not in the string")
#
# index() does the same thing but throws an exception if not found
#
n = tststr.rindex(lookfor)
print(n)
```

find() Finds string and returns index (0 based)

-1 if not found

args: value, start, end

index() Same as find() but raises exception if not found

rfind() Finds last location of string and returns index (0 based)

-1 if not found

args: value, start, end

rindex() Same as rfind() but raises exception if not found

find() Finds string and returns index (0 based)

-1 if not found

args: value, start, end

index() Same as find() but raises exception if not found

rfind() Finds last location of string and returns index (0 based)
-1 if not found
args: value, start, end
rindex() Same as **rfind()** but raises exception if not found

Python Code: 8_str_replace.py

```
#  
#   Replacing a substring with number of occurrences to be replaced  
#  
tststr = "The quick brown fox jumped over the lazy dog"  
tst2 = tststr.replace("o", "*")  
print(tststr, tst2)  
#  
#   Expanding tabs  
#  
tststr="The\tquick\tbrown fox"  
print(tststr, tststr.expandtabs(1))
```

replace()

Returns a string where a certain substr is replaced with a new substr
args: oldvalue, newvalue, count (default = all occurrences)

expandtabs()

Returns a string where tabs are replaced by number of spaces specified
args: number of spaces used to expand \t (default = 8)

Python Code: 9_str_partition.py

```
tststr = "In the forest the quick brown fox jumped over the lazy dog"  
result=tststr.rpartition("the")  
print(result)
```

partition(argstring)

Looks for first occurrence of argstring

Returns a 3-element tuple containing

The string preceding the argstring

The argstring itself

The string after the argstring

If not found, returns a tuple containing the original string and two empty strings

rpartition(argstring)

Looks for last occurrence of argstring

Returns a 3-element tuple containing

The string preceding the argstring

The argstring itself

The string after the argstring

If not found, returns a tuple containing two empty strings and the original string

Python Code: 10_str_split (1).py

```
#
#   Splitting a string into a list
#
tststr = "The quick\nbrown fox\njumped over\nthe lazy dog"
lstlines=tststr.splitlines()
print(lstlines)
lstlines=tststr.splitlines(True)
print(lstlines)
lstlines=tststr.split()
print(lstlines)
lstlines=tststr.split("he")
print(lstlines)
lstlines=tststr.split("he",1)
print(lstlines)
lstlines=tststr.rsplit()
print(lstlines)
lstlines=tststr.rsplit("he")
print(lstlines)
lstlines=tststr.rsplit("he",1)
print(lstlines)
```

Python Code: 10_str_split.py

```
#
#   Splitting a string into a list
#
tststr = "The quick\nbrown fox\njumped over\nthe lazy dog"
lstlines=tststr.splitlines()
print(lstlines)
lstlines=tststr.splitlines(True)
print(lstlines)
lstlines=tststr.split()
print(lstlines)
lstlines=tststr.split("he")
print(lstlines)
lstlines=tststr.split("he",1)
print(lstlines)
lstlines=tststr.rsplit()
print(lstlines)
lstlines=tststr.rsplit("he")
print(lstlines)
lstlines=tststr.rsplit("he",1)
print(lstlines)
```

`splitlines(keeplinebreaks)`

Breaks string into a LIST where each line is a list item

Arg=True => keep linebreaks. Default = False

`split(separator,maxsplit)`

Breaks string into a LIST where each word is a list item

`rsplit(separator,maxsplit)`

Starting from right, breaks string into a LIST

separator defines list items (default = space). Maxsplit = no. of splits to do. Number of list items = Maxsplit + 1. Default=all.

`splitlines(keeplinebreaks)`

Breaks string into a LIST where each line is a list item

Arg=True => keep linebreaks. Default = False

`split(separator,maxsplit)`

Breaks string into a LIST where each word is a list item

`rsplit(separator,maxsplit)`

Starting from right, breaks string into a LIST

separator defines list items (default = space). Maxsplit = no. of splits to do. Number of list items = Maxsplit + 1. Default=all.

Python Code: 11_str_join (1).py

```
#
#   Joining lists and tuples into a string
#
tstlist = ['The quick', 'brown fox', 'jumped over', 'the lazy dog']
res_string = "-----".join(tstlist)
print(res_string)
tsttup = ('The quick', 'brown fox', 'jumped over', 'the lazy dog')
res_string = "***".join(tsttup)
print(res_string)
```

Python Code: 11_str_join.py

```
#
#   Joining lists and tuples into a string
#
tstlist = ['The quick', 'brown fox', 'jumped over', 'the lazy dog']
res_string = "-----".join(tstlist)
print(res_string)
tsttup = ('The quick', 'brown fox', 'jumped over', 'the lazy dog')
res_string = "***".join(tsttup)
print(res_string)
```

Special String methods

`sep.join(iterable)`

Called using a separator string. Returns a string consisting of all elements of the iterable (elements must be str), joined by separator

Python Code: 12_str_fstrings.py

```
#  
# The power of f-strings  
#  
num1 = 100  
list1 = ["List element", 101, True]  
tuple1 = (1,2,3)  
fstring = f"f-string containing a number: {num1}, a list: {list1} and a tuple:  
{tuple1}"  
print(fstring)
```

Python Code: 14_str_fstr_width.py

```
num=100  
lnum=1234  
fnum=123.456  
print(f"An f-sring with width=10: {num:10}")  
print(f"An f-sring with width=10: {lnum:10}")  
print(f"An f-sring with width=10 and 4 decimal places: {fnum:10.4f}")  
print(f"An f-sring with width=10 aligned left: {num:<10}") # same as no width  
specifier  
print(f"An f-sring with width=10 aligned right: {num:>10}")  
print(f"An f-sring with width=10 aligned right and padded: {num:*>10}")  
print(f"An f-sring with width=10 aligned left and padded: {num:*<10}")  
print(f"An f-sring with width=10 aligned right and rounded to 2 places:  
{fnum:.2f}")
```

Special String methods (contd.)

f strings

f"string with {optional formatting} placeholders"

Simple method to construct complicated strings for printing

Formatting is mainly used with numeric and date types

Very rich formatting options

f strings (contd.)

f"string {var} string {var}...."

Automatically transforms var to strings and concatenates

Add = to print the name of the variable AND its value

E.g. f"string {var=}" for debugging

For numbers: {var:[width][.precision][type]}

Where type=d or f (int or float). Precision specified only for float

Alignment:

Right: {var:>[width]}

Left: {var:<[width]}

Padding:

Pad right with char for width: {var:char>[width]}

Pad left with char for width: {var:char<[width]}

Rounding to n decimal places:

{var:.nf}

Python Code: 0_str_substr.py

```
tststr = "The quick brown fox"
#
#   Membership operators
#
print("quick" in tststr)
print("quick" not in tststr)
#
#   String slicing using index
#
substr = tststr[5:11]
print(substr)
reststr = tststr[5:]
print(reststr)
leftstr = tststr[:5]
print(leftstr)
#
#   Indexing from the end. End not included
#
endstr = tststr[-5:-1]
print(endstr)
endstr = tststr[-5:len(tststr)]
print(endstr)
#
#   String concatenation
#
newstr = tststr + tststr
print(newstr)
```

Checking Substring with membership

Substr in string

Substr not in string

Returns True or false

String concatenation = addition of two strings

newstr = str1 + str2

Slicing

Using indexing for slicing a string

newstring = oldstring[from:to] (ending index is NOT included)

newstring = oldstring[from:] (to the end)

newstring = oldstring[:to] (from the beginning, NOT inc to)

Slicing from the end:

Last index is -1

Can index backwards

e.g. newstring = oldstring[-5:-1] (last index is NOT included)

Python Code: 1_str_type_methods.py

```
# tststr = "Abc123"
tststr = "abcdef" # Superscript
print(tststr)
print(f"{tststr.isalnum()}") # True if alphanumeric string
print(f"{tststr.isalpha()}") # True if alphabetic string
print(f"{tststr.isascii()}") # True if ascii string
print(f"{tststr.isdecimal()}") # True if all characters are decimals
print(f"{tststr.isdigit()} (includes sub/super scripts)") # True if
all characters are digits
print(f"{tststr.isnumeric()} (includes sub/super scripts)") # True
if all characters are digits
print(f"{tststr.isidentifier()}") # True if can be the name of a
variable
print(f"{tststr.islower()}") # True if all characters are lower
case
print(f"{tststr.isupper()}") # True if all characters are
uppercase
print(f"{tststr.isprintable()}") # True if all characters are printable
print(f"{tststr.isspace()}") # True if all characters are white
space
print(f"{tststr.istitle()}") # True if string print(f"{tststr.is
title case
```

Determining the type of a string

isalnum()	True if alphanumeric string
isalpha()	True if alphabetic string
isascii()	True if ascii string
isdecimal()	True if all characters are decimals
isdigit()	True if all characters are digits
isnumeric()	True if all characters are digits
isidentifier()	True if can be the name of a variable
islower()	True if all characters are lower case
isupper()	True if all characters are uppercase
isprintable()	True if all characters are printable

`isspace()` True if all characters are white space
`istitle()` True if string is title case

Python Code: 2_lists.py

```
#
#   List creation
#
alist = [1, 'Fox', 3]
blist = list((1, 'Fox', 3))    # list from tuple using constructor
clist = list()                # empty list
print(f"{alist = }")
print(f"{blist = }")
print(f"{clist = }")
#
#   Duplicate elements allowed
#
dlist = [1, 1, 2, 3]
print(f"{dlist = }")
```

Lists

One of four Python data types used to store collections

(Others are tuples, sets, dictionaries)

Created by using []

Can contain any type(s) of data – mixed items allowed

May also be created using `list()` constructor (`arg=iterable`)

Allow duplicate members.

Python Code: 3_lists_indexing.py

```
#
#   Lists are ordered, indexed and changeable
#
mylist = ["House 123", "Lane 5", "Peshawar Road", "Rawalpindi", 54600]
print(mylist, type(mylist), len(mylist))

for listel in mylist:
    print(listel)

for i in range(len(mylist)):
    print(mylist[i])

i = 0
while i < len(mylist):
    print(mylist[i])
    i = i + 1

mylist[2] = 300
print(mylist)
```

Are ordered and changeable.

Iterating through lists

Can use the “for” and “while” loops

Any element can be changed (type is irrelevant)

Applicable Python functions: type(), len(), list()

Python Code: 4_list_slicing.py

```
mylist = ["The", "quick", "brown", "fox"]
#
#   Membership operators
#
print("quick" in mylist)
print("quick" not in mylist)
#
#   List slicing using index
#
sublist = mylist[1:3]
print(sublist)
restlist = mylist[2:]
print(restlist)
leftlist = mylist[:3]
print(leftlist)
#
#   Indexing from the end. End not included
#
endlist = mylist[-3:-1]
print(endlist)
endlist = mylist[-3:]
print(endlist)
#
#   list concatenation
#
newlist = mylist + mylist
print(newlist)
```

Can use range of indices

Like strings, indices go from 0 to len(list) – 1

Last element has index = -1

Negative indexing is permitted

Behavior is exactly like strings

Python Code: 5_list_changing.py

```
#
#   Mutable lists. A value can be changed
#
fruitlist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
print(fruitlist)
fruitlist[1] = "blackcurrant"
print(fruitlist)
#
#   Changing range of list values
#
```

```

fruitlist[1:3] = ["blackcurrant", "watermelon"]
print(fruitlist)
#
#   Change the second value by replacing it with two new values. The list
expands:
#
fruitlist[1:2] = ["apricot", "falsa"]
print(fruitlist)
#
#   Replace the 2nd & 3rd values by a single one. The list shrinks:
#
fruitlist[1:3] = ["kinnow"]
print(fruitlist)

```

Substr in string

Substr not in string

Returns True or false

String concatenation = addition of two strings

```
newstr = str1 + str2
```

Using indexing for slicing a string

```
newstring = oldstring[from:to] (ending index is NOT included)
```

```
newstring = oldstring[from:] (to the end)
```

```
newstring = oldstring[:to] (from the beginning, NOT inc to)
```

Slicing from the end:

Last index is -1

Can index backwards

e.g. `newstring = oldstring[-5:-1]` (last index is NOT included)

`isalnum()` True if alphanumeric string

`isalpha()` True if alphabetic string

`isascii()` True if ascii string

`isdecimal()` True if all characters are decimals

`isdigit()` True if all characters are digits

`isnumeric()` True if all characters are digits

`isidentifier()` True if can be the name of a variable

`islower()` True if all characters are lower case

`isupper()` True if all characters are uppercase

`isprintable()` True if all characters are printable

`isspace()` True if all characters are white space

`istitle()` True if string is title case

One of four Python data types used to store collections

(Others are tuples, sets, dictionaries)

Created by using `[]`

Can contain any type(s) of data – mixed items allowed

May also be created using list() constructor (arg=iterable)

Allow duplicate members.

Are ordered and changeable.

Iterating through lists

Can use the “for” and “while” loops

Any element can be changed (type is irrelevant)

Applicable Python functions: type(), len(), list()

Can use range of indices

Like strings, indices go from 0 to len(list) - 1

Last element has index = -1

Negative indexing is permitted

Behavior is exactly like strings

Inserting / changing list elements

Mutable – unlike strings

May replace, insert or remove elements by using indexing

Lists expand and contract according to original and replacement items

List methods

Adding elements and extending lists

insert(position, value)

append(value)

extend(iterable)

iterable = any Python iterable

Deleting elements

remove(value)

pop(optional index else last)

clear()

del list[index]

del list

List comprehension

Shortcut for creating lists based on condition

newlist = [expression for item in iterable if condition == True]

Very powerful construct

List sorting

Lists can be sorted using

```
sort()
sort(reverse = True)
sort(key = function)
```

Copying lists

List2 = list1 does not work!

Simply creates a reference to the original list

Need to do a deep copy

Alternatives are:

```
copy() method
list() constructor method based on any iterable
using the slice operator [:] which fools the direct copy
```

Python Code: 6_list_insertion.py

```
#
# List methods
#
# Inserting values without replacing. Use the insert() method
#
fruitlist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
print(fruitlist)
fruitlist.insert(2, "Fox")
print(fruitlist)
#
# Using the append() method to append an item:
#
fruitlist.append("Kinnow")
print(fruitlist)
#
# Extending a list by adding another list
#
dryfruits = ["Almonds", "Pistachios", "Walnuts"]
fruitlist.extend(dryfruits)
print(fruitlist)
#
# Can be extended using any other iterable
#
fruitlist.extend("This") # using string as iterable. insert characters
print(fruitlist)
fruitlist.extend((1,2,3)) # using tuple to extend. Each value becomes list
element
print(fruitlist)
```

Adding elements and extending lists

`insert(position, value)`

`append(value)`

`extend(iterable)`

iterable = any Python iterable

Python Code: 7_list_del_element.py

```
#
# List methods
#
# Deleting values . Use the remove() method
#
fruitlist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
print(fruitlist)
fruitlist.remove("orange")
print(fruitlist)
#
# Only the first occurrence is removed
#
fruitlist.extend(fruitlist)
print(fruitlist)
fruitlist.remove("apple")
print(fruitlist)

#
# Removing by index: pop()
#
fruitlist.pop(3)
print(fruitlist)
fruitlist.pop()      # No index specified. Last element removed
print(fruitlist)
#
# Emptying a list
#
fruitlist.clear()
print(fruitlist)      # List is now empty.
#
# Populate it again:
#
fruitlist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
print(fruitlist)
#
# Deleting an element using index or a range of indices
#
del fruitlist[0]
print(fruitlist)
del fruitlist[0:2]
print(fruitlist)
#
# Delete entire list
#
del fruitlist
#
```

```
# Now we will get an error because the list no longer exists
#
print(fruitlist)
```

Deleting elements

```
remove(value)
pop(optional index else last)
clear()
```

```
del list[index]
del list
```

Python Code: 8_list_comprehension.py

```
#
# List comprehension
#
fruitlist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
#
# Create a new list for fruits without an o in their name. First, with loops:
myfruits1 = []
for x in fruitlist:
    if "o" not in x:
        myfruits1.append(x)
print(f"{fruitlist = }")
print(f"{myfruits1 = }")
#
# Now using list comprehension with expression x+x
#
myfruits2 = [x for x in fruitlist if "o" not in x] # Just one line of code!
print(f"{myfruits2 = }")
#
# Specifying a condition is not necessary
#
myfruits3 = [x+x+x for x in fruitlist] # Just one line of code!
print(f"{myfruits3 = }")
#
# Using an iterable to generate a list
#
myecho = ["Hello." for x in range(5)]
print(f"{myecho = }")
#
# Or using the number of elements in another iterable
#
myecho = ["Hello World" for x in fruitlist]
print(f"{myecho = }")
#
# Creating a list of numbers divisible by 7
#
mylist = [x for x in range(100) if x%7 == 0]
print(f"{mylist = }")
#
```

```
# Conditions can also be applied to the expression before adding to list
#
mylist = [x if x<50 else 0 for x in range(100) if x%7 == 0]
print(f"{mylist = }")
```

Shortcut for creating lists based on condition

`newlist = [expression for item in iterable if condition == True]`

Very powerful construct

Python Code: 9_list_sort.py

```
#
#   Sorting lists. Elements must be the same data type
#
fruitlist = ['banana', 'cherry', 'apple', 'orange', 'kiwi', 'mango']
print(f"List before sorting: {fruitlist}")
fruitlist.sort()
print(f"List after sorting: {fruitlist}")
fruitlist.sort(reverse=True)
print(f"List after reverse sorting: {fruitlist}\n")
#
#   Case insensitive sort
#
fruitlist = ['Banana', 'cherry', 'apple', 'orange', 'kiwi', 'mango']
fruitlist.sort()
print(f"List after sorting: {fruitlist}")
fruitlist.sort(key = str.lower)
print(f"List after sorting with key: {fruitlist}\n")
#
#   Numerical sorts
#
mynums = [10, 9, 8, - 11, -12, -13]
print(f"Numerical list: {mynums}")
mynums.sort()
print(f"Numerical list after sorting: {mynums}\n")
mynums.sort(reverse = True)
print(f"Numerical list after reverse sorting: {mynums}")
mynums.sort(key = abs)
print(f"Numerical list after sorting on absolute value: {mynums}")

def myfunc(n):
    return 100-abs(n)

mynums.sort(key = myfunc)
print(f"Numerical list after sorting using a function:\n {mynums}")
```

Lists can be sorted using

`sort()`

```
sort(reverse = True)
sort(key = function)
```

Python Code: 10_list_copy.py

```
#
#   Copying lists
#
fruitlist1 = ['banana', 'cherry', 'apple', 'orange', 'kiwi']
fruitlist2 = fruitlist1
fruitlist1[0] = "watermelon"
print(f"Shallow: {fruitlist1 = }")
print(f"          {fruitlist2 = }\n")    # No good. Shallow copy
#
#   Using the copy method
#
fruitlist1 = ['banana', 'cherry', 'apple', 'orange', 'kiwi']
fruitlist2 = fruitlist1.copy()
fruitlist1[0] = "watermelon"
print(f"Copy method: {fruitlist1 = }")
print(f"Second list: {fruitlist2 = }\n")
#
#   Using the list() constructor
#
fruitlist1 = ['banana', 'cherry', 'apple', 'orange', 'kiwi']
fruitlist2 = list(fruitlist1)
fruitlist1[0] = "watermelon"
print(f"List constructor: {fruitlist1 = }")
print(f"      Second list: {fruitlist2 = }\n")
#
#   Using the slice operator
#
fruitlist1 = ['banana', 'cherry', 'apple', 'orange', 'kiwi']
fruitlist2 = fruitlist1[:]
fruitlist1[0] = "watermelon"
print(f"Slice operator: {fruitlist1 = }")
print(f"      Second list: {fruitlist2 = }")
```

List2 = list1 does not work!

Simply creates a reference to the original list

Need to do a deep copy

Alternatives are:

- copy() method
- list() constructor method based on any iterable
- using the slice operator [:] which fools the direct copy

Python Code: 0_tuples.py

```
#
#   Tuple creation
#
atuple = (1, 'Fox', 3)
btuple = tuple([1, 'Fox', 3])    # tuple from tuple using constructor
```

```

ctuple = tuple()                # empty tuple
dtuple = ("single",)
print(f"{atuple = }")
print(f"{btuple = }")
print(f"{ctuple = }")
print(f"{dtuple = }")#
#   Duplicate elements allowed
#
dtuple = (1, 1, 2, 3)
print(f"{dtuple = }")

```

Tuples

One of four Python data types used to store collections

(Others are lists, sets, dictionaries)

Created by using ()

if defining using a single element, must end with comma

Can contain any type(s) of data – mixed items allowed

May also be created using tuple() constructor (arg=iterable)

Allow duplicate members.

Python Code: 1_tuple_indexing.py

```

#
#   Tuples are ordered, indexed and NOT changeable
#
mytuple = ("House 123", "Lane 5", "Peshawar Road", "Rawalpindi", 54600)
print(mytuple, type(mytuple), len(mytuple))

for tupleel in mytuple:
    print(tupleel)

for i in range(len(mytuple)):
    print(mytuple[i])

i = 0
while i < len(mytuple):
    print(mytuple[i])
    i = i + 1

mytuple[2] = 300 # This will throw an error.A tuple is immutable

```

Tuple indexing

Tuples are ordered and NOT changeable.

Iterating through tuples

Can use the “for” and “while” loops

No element can be changed

Python Code: 2_tuple_slicing.py

```
mytuple = ("The", "quick", "brown", "fox")
#
#   Membership operators
#
print("quick" in mytuple)
print("quick" not in mytuple)
#
#   tuple slicing using index
#
subtuple = mytuple[1:3]
print(subtuple)
resttuple = mytuple[1:]
print(resttuple)
lefttuple = mytuple[:3]
print(lefttuple)
#
#   Indexing from the end. End not included
#
endtuple = mytuple[-3:-1]
print(endtuple)
endtuple = mytuple[-3:]
print(endtuple)
#
#   tuple concatenation
#
newtuple = mytuple + mytuple
print(newtuple)
```

Tuple slicing

Can use range of indices

Like strings and lists, indices go from 0 to $\text{len}(\dots) - 1$

Last element has index = -1

Negative indexing is permitted

Behavior is exactly like lists

Python Code: 3_tuple_changing.py

```
#
#   Tuples are immutable. A value cannot be changed
#
fruittuple = ("apple", "banana", "cherry", "orange", "kiwi", "mango")
print(fruittuple)
#
#   One workaround would be to convert tuple to a list, change the list and
#   make it a tuple again.
#
fruitlist=list(fruittuple)
fruitlist[1] = "blackcurrant"
```

```

fruittuple = tuple(fruitlist)
print(fruittuple)
#
# Using this technique, all of the list changing methods can be applied to
tuples
#
# Exception: Tuples can be extended by another tuple by using assignment
operator
#
dryfruittuple=("Almonds","Pistachios","Walnuts")
fruittuple += dryfruittuple # works because LHS becomes a new definition
print(fruittuple)

```

Tuple changing

Cannot be changed

Workaround:

Convert tuple to a list

Make changes

Return to tuple

Elements CANNOT be added or removed

However, a tuple can be extended by adding another tuple

(The same applies to strings which are also immutable)

Python Code: 4_tuple_unpacking.py

```

#
# Unpacking tuples
#
# Defining a tuple is called "packing"
#
fruittuple = ("apple", "banana", "cherry", "orange", "kiwi", "mango")
print(fruittuple)
#
# A tuple can be unpacked by providing the correct number of variables on the
LHS
#
f1, f2, f3, f4, f5, f6 = fruittuple
print(f"{f1=}, {f2=}, {f3=}, {f4=}, {f5=}, {f6=}")
#
# If the number of elements on the LHS are less, using * will put the
# remaining into a list
#
f1, f2, f3, *f4 = fruittuple
print(f"{f1=}, {f2=}, {f3=}, {f4=}")
f1, f2, *f3, f4 = fruittuple
print(f"{f1=}, {f2=}, {f3=}, {f4=}")
*f1, f2, f3, f4 = fruittuple
print(f"{f1=}, {f2=}, {f3=}, {f4=}")

```

Packing and Unpacking Tuples

Defining a tuple is called “packing”

However, we can also unpack a tuple

Provide the correct number of variables on the LHS

(The same can be done with lists and strings!)

If the number of variables does not match, use * for the last variable (or any variable and the remaining variables will automatically be assigned the correct values)

Python Code: 5_python_consistency_lists_strings.py

```
#
# Python is very consistent
# A single line of code can do a lot!
#
# Unpacking lists and strings
#

fruitlist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
print(fruitlist)
#
#   A list can be unpacked by providing the correct number of variables on the
LHS
#
f1, f2, f3, f4, f5, f6 = fruitlist
print(f"{f1=}, {f2=}, {f3=}, {f4=}, {f5=}, {f6=}")
#
#   If the number of elements on the LHS are less, using * will put the
remaining into a list
#
f1, f2, f3, *f4 = fruitlist
print(f"{f1=}, {f2=}, {f3=}, {f4=}")
f1, f2, *f3, f4 = fruitlist
print(f"{f1=}, {f2=}, {f3=}, {f4=}")
*f1, f2, f3, f4 = fruitlist
print(f"{f1=}, {f2=}, {f3=}, {f4=}")
#
# Unpacking strings
#
stdstr = "MyString"
print(stdstr)
#
#   A string can be unpacked by providing the correct number of variables on
the LHS
#
f1, f2, f3, f4, f5, f6, f7, f8 = stdstr
print(f"{f1=}, {f2=}, {f3=}, {f4=}, {f5=}, {f6=}, {f7=}, {f8=}")
#
#   If the number of elements on the LHS are less, using
#   * will put the remaining into a list with each element = single char string
#
f1, f2, f3, *f4 = stdstr
```

```

print(f"{f1=}, {f2=}, {f3=}, {f4=}")
f1, f2, *f3, f4 = stdstr
print(f"{f1=}, {f2=}, {f3=}, {f4=}")
*f1, f2, f3, f4 = stdstr
print(f"{f1=}, {f2=}, {f3=}, {f4=}")

```

Python consistency

Unpacking also applies to

Lists

Strings

Allows for writing very concise code

Python Code: 6_tuple_adding_multiplying.py

```

#
# Adding tuples
# and multiplying!
#
firsttuple = (1,2,3)
secondtuple = (4, 5, 6)
thirtuple = firsttuple + secondtuple
fourtuple = 2 * firsttuple
print(f"{firsttuple=}, \n{secondtuple=}, \n{thirtuple =}, \n{fourtuple =}")
#
# Agin, Python is consistent. Applies to lists and strings as well
#
firstlist = [1,2,3]
secondlist = [4, 5, 6]
thirdlist = firstlist + secondlist
fourthlist = 2 * firstlist
print(f"{firstlist=}, \n{secondlist=}, \n{thirdlist =}, \n{fourthlist =}")
#
# Finally, strings
#
firststr = "The "
secondstr = "quick"
thirdstr = firststr + secondstr
fourthstr = 2 * firststr
print(f"{firststr=}, \n{secondstr=}, \n{thirdstr =}, \n{fourthstr =}")

```

Adding and multiplying tuples

Two tuples can be added to give a third

Multiplying a tuple with an integer gives a new tuple

The same applies to lists and strings - consistent

Python Code: 7_tuple_methods.py

```
#
#   Tuple methods
#
#   Count() and index()
#
mytuple = ( 1, 2, 3, 1, 2, 3, 1, 2)
print(f"{mytuple =}")
print(f"Count:\nThe number 1 occurs {mytuple.count(1)} times")
print(f"The number 3 occurs {mytuple.count(3)} times")
print(f"Index:\nThe number 2 occurs at index {mytuple.index(2)}")
#
#   And being consistent, Python functions len, type and del apply:
#
print(f"The type of mytuple is {type(mytuple)}")
print(f"The length of mytuple is {len(mytuple)}")

del mytuple # No longer exists
print(mytuple) # Will cause an error
```

Tuple methods

Count()

mytuple.count(value)

Index()

mytuple.index(value)

Applicable Python functions:

type(), len(), del

Python Code: 0_sets.py

```
#
#   Creating a set using {}
#
set1={1,2,3,4}
print(f"A set created using braces: {set1 = }")
set2={"a","b", 100}
print(f"A set created using braces: {set2 = }\n")
#
#   Creating an empty set using set() constructor. Cannot use {} due to dict
#
set3=set()
print(f"An empty set created using the constructor: {set3 = }\n")
#
#   Creating a set using the set constructor on an iterable
#
```

```
list1 = [1, 2, 3, "Fox"]
set4 = set(list1)
print(f"A set constructed using the constructor on a list:\n\t{list1}=\n\t{set4}
=}")
```

Sets

One of four Python data types used to store collections

(Others are lists, tuples, dictionaries)

Created by using set(), or set(iterable)

Can be created using {} but not an empty set since {} = dict

Can contain any type(s) of data – mixed items allowed

Unordered, unindexed, items are unchangeable

No duplicates allowed!

Python Code: 1_sets_iterating.py

```
#
# Sets are unordered and unindexed. Hence only membership works for checking
# an element
#
set1 = {1, 2, 3, "dog", "cat", "owl"}
for x in set1:
    if x == "cat":
        print(f"Found the cat! {x}")
    else:
        print(f"Not the cat: {x}")
#
# Logical tests use the membership operator as well
#
if "owl" in set1 and "cat" in set1:
    print("We have found a cat and an owl!")
#
# Iterating through a set can only be done with the membership operator
#
set2 = {"the", 100, "quick", 200, "brown", 300}
print(f"\nIterating over a set, elements appear in random order:")
for x in set2:
    print(x)
```

Accessing set items

Unordered and unindexed so no direct access to set items

The membership operator applies to sets

Loop through sets using the membership operator only

Or used to determine the presence or absence of an element

No element can be changed

Python Code: 2_set_fine_points.py

```
#
# Duplicate values are ignored
```

```

#
set1 = {1,2,3,1,2,3,4}
print(f"Duplicate values have been ignored: {set1 = }\n")
#
# 1 and True are identical, hence duplicates are removed
#
set2 = {1, "Fox", True, "Good morning"}
print(f"Only one out of 1 and True is kept: {set2 = }\n")
#
# (1, True) treated differently from (False, 0): whichever comes first
#
set3 = {1, "Fox", True, "Good morning", False, 0, "dog"}
print(f"Again, either 0 or False is kept: {set3 = }\n")
#
# We did not see a 0 in set3. Does it exist?
#
if 0 in set3:
    print("0 exists but prints as False!\n")
#
# Finally, the usual Python functions apply:
#
print(f"{set3 = }\n{type(set3)=}, {len(set3)=}\n")

```

Some fine points

No duplicates allowed. If defined, will be ignored.

True == 1 and False == 0. Treated identically

Membership test can be done with either!

The usual Python functions apply:

type() len() and the constructor set()

Python Code: 3_sets_adding.py

```

set1 = {1,2,3}
print(f"{set1 =}")
#
# Add a new value to the set. Duplicates will be ignored
#
set1.add(100)
print(f"After adding a new value: 100, {set1 = }\n")      # Unordered print
#
# Augment a set with another full set using the update() method
#
set1 = {1, 2, 3}
set2 = {"the", "quick", "fox"}
tuple1 = (11, 22, 33)
list1 = [111, 222, 333]
#
# Update method changes original set
#
print(f"{set1 =}")
set1.update(set2)
print(f"set1 updated with the set {set2} \nchanges set1 to: {set1}\n")

```

```
#
# Update can take multiple args of type iterable
#
set1.update(list1, tuple1)
print(f"set1 updated with list {list1} \nand tuple {tuple1} \nchanges set1 to:
\n{set1 =}\n")
#
# The following update changes nothing since duplicates are ignored/removed
#
set1.update(list1, tuple1)
print(f"Updated again with the same iterables changes nothing: \n{set1 =}\n")
```

Adding to and updating sets

A set, once created, cannot have any element changed

BUT:

Elements can be added or removed!

A set can be changed, not an element!

Use add() to add a single element or update(iterable) to extend

Python Code: 4_sets_removing.py

```
set1 = {1, 2, 3, 100, 'Dog', 200, 'quick', 300, 'fox', 'Lazy', 'brown'}
print(f"{set1 = }\n")
set1.remove("Dog")
print(f"After removing the value Dog: \n{set1 = }\n")
set1.discard("Dog")      # No error if value not found. Remove will give an
error
print(f"After discarding the value Dog again - no error: \n{set1 = }\n")
#set1.remove("Dog")      # Throws a KeyError
a = set1.pop()           # A random element is removed and returned
print(f"The value removed is '{a}', and the remaining set is \n{set1}\n")
#
# Emptying a set using clear()
#
set1.clear()
print(f"set1 after the clear() method is called: {set1}\n")
#
# The del function removes the set from memory
#
del set1
# print(set1)            # Will now give an error
```

Removing elements

We can use remove elements by
remove(), discard(), pop() methods

Emptying a set:

`clear()` method

Deleting a set from memory:

`del` function

Python Code: 5_sets_union.py

```
set1 = {1, 2, 3}
set2 = {"the", "quick", "fox"}
#
# Using union to join two sets
#
set3 = set1.union(set2)
print(f"{set1 =},\n{set2 =},\nand their union: {set3 =}\n")
#
# Set3 union Set1 results in nothing new since all elements are duplicates
#
set4 = set3.union(set1)
print(f"set3 union set1 gives nothing new: {set4 =}\n")      # No change so
set4 == set3
#
# Union can be with mutiple sets
#
set5 = {False, 100}
set6 = set1.union(set2, set5)
print(f"The union of {set1} with {set2} and {set5} gives:\n{set6}\n")
#
# Union can be with other iterables
#
tuple1 = (11, 22, 33)
list1 = [111, 222, 333]
set7 = set1.union(tuple1, list1)
print(f"The union of {set1} with the tuple {tuple1} and list {list1}
gives:\n{set7}\n")
#
# Shorthand for union is the | operator, but only works with sets
#
set8 = set2 | set5
print(f"{set2 =}\nunion\n{set5 =}\nusing the | operator (only works with sets)
gives:\n{set8 = }")
```

Set methods

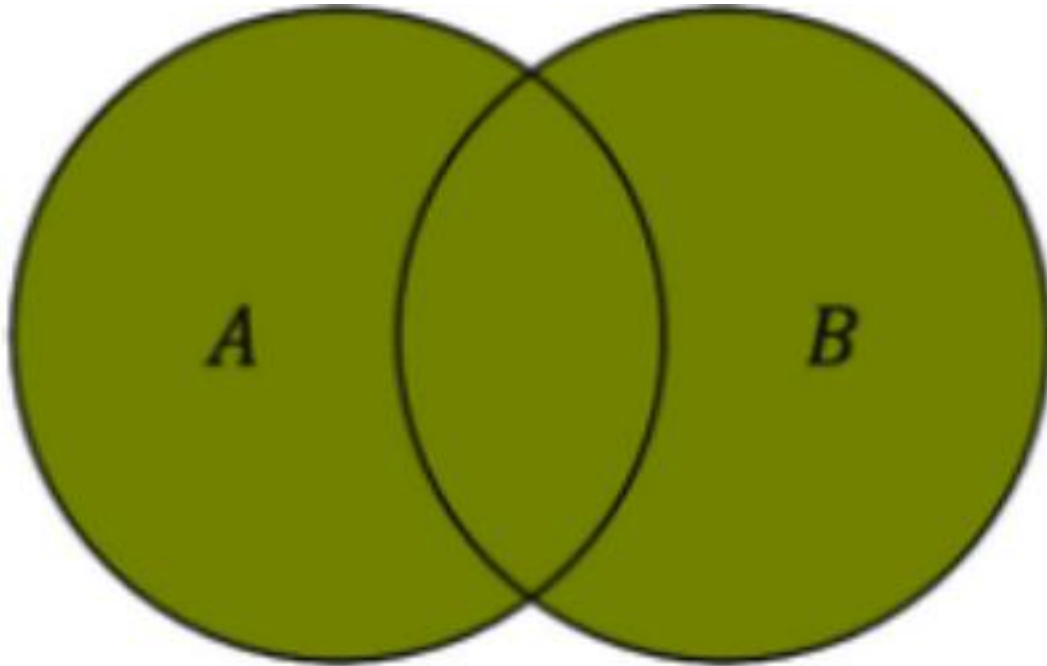
`Union()`

Returns a new set

Can take 1 or more args

Args can be a mix of iterables

Shorthand is the `|` operator, but works ONLY with sets



Python Code: [6_sets_intersection.py](#)

```
#
#   The intersection method returns a new set with elements that are present in
both sets
#
set1 = {1,2,3}
set2 = {1,3,5,7}
set3 = set1.intersection(set2)
set4 = set2.intersection(set1)
print(f"The intersection method returns a new set containing common elements
only. Thus:\n\t{set1}=\n\tintersection\n\t{set2}=\ngives\n\t{set3}=\n")
print(f"And identically:\n\t{set2}=\n\tintersection\n\t{set1}=\ngives\n\t{set4}
=\n")
#
#   The intersection can be done with some other iterables!
#
list1 = ["The","quick","brown",1,11,111]
set5 = set1.intersection(list1)
print(f"A set\n\t{set1}=\n\tintersection with a list\n\t{list1}
=\ngives\n\t{set5}=\n")
#
#   The intersection can be done with multiple iterables!
#
set6 = set1.intersection(list1,set3)
print(f"A set\n\t{set1}=\n\tintersection with a list\n\t{list1}=\nand a
set\n\t{set3}=\ngives\n\t{set6}=\n")
#
#   And remember that 1=True and 0=False in sets:
#
set1 = {"apple", 1, "banana", 0, "cherry"}
set2 = {False, "google", True, "apple", 2}
set7 = set1.intersection(set2)
```

```

print(f"Intersection of sets with True/False values\n\t{set1 =}\nintersection
with\n\t{set2 =}\ngives\n\t{set7 =}\n")
#
#   Finally, intersection_update() changes the calling set:
#
set1.intersection_update(set2)
print(f"Intersection update 'set1.intersection_update(set2)' changes the
calling set1:\n\t{set1 =}\n")

```

`intersection()`

Method returns a new set with values that are common

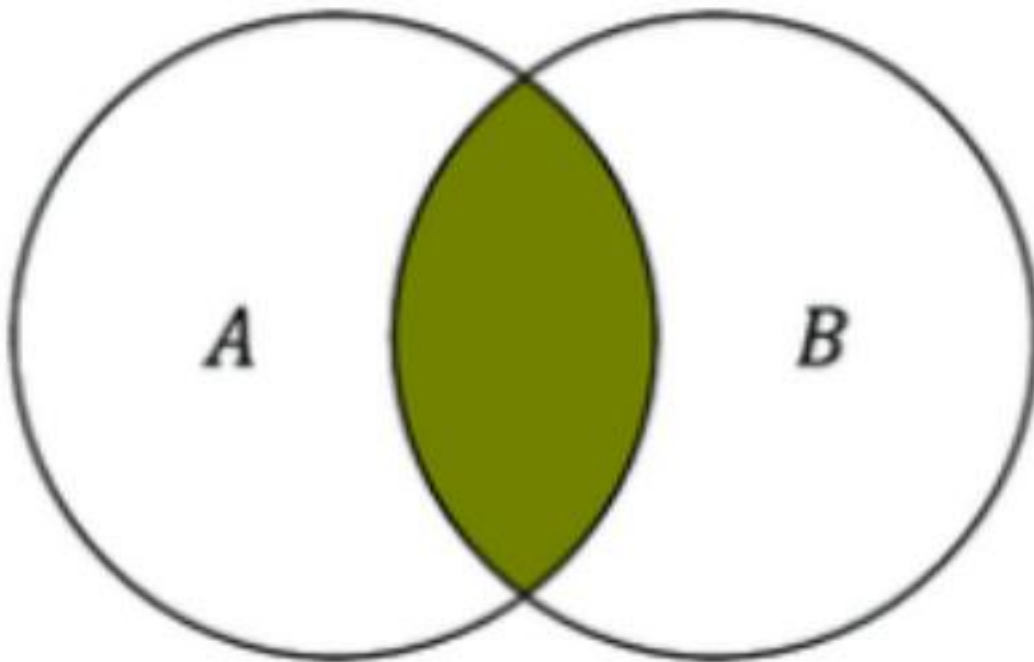
The method can work with other iterables as well

The method can take multiple arguments

Shorthand is the & operator, but works only with sets

`intersection_update()`

Changes the calling set



Python Code: [7_sets_difference.py](#)

```

#
#   The difference method returns a new set with elements that are only present
in the calling set
#

```

```

set1 = {1,2,3}
set2 = {1,3,5,7}
set3 = set1.difference(set2)
set4 = set2.difference(set1)
print(f"The difference method returns a new set containing values present ONLY
in the calling set. Thus:\n\t{set1 =}\ndifference\n\t{set2 =}\ngives\n\t{set3
=}\n")
print(f"And interestingly:\n\t{set2 =}\ndifference\n\t{set1 =}\ngives\n\t{set4
=}\nNo symmetry!\n")
#
#   The difference can be done with some other iterables!
#
list1 = ["The", "quick", "brown", 1, 11, 111]
set5 = set1.difference(list1)
print(f"A set\n\t{set1 =}\ndifference with a list\n\t{list1 =}\ngives\n\t{set5
=}\n")
#
#   The difference can be done with multiple iterables!
#
set6 = set1.difference(list1, set3)
print(f"A set\n\t{set1 =}\ndifference with a list\n\t{list1 =}\nand a
set\n\t{set3 =}\ngives\n\t{set6 =}\n")
#
#   difference_update() changes the calling set:
#
print(f"Difference_update() changes the calling set. Thus\n\t{set1
=}\ndifference_update\n\t{set2 =}\ngives")
set1.difference_update(set2)
print(f"\t{set1 =}\n")
#
#   symmetric_difference() returns a new set with no common elements:
#
set1 = {"apple", "banana", "cherry"}
set2 = {"samsung", "redmi", "apple"}

set3 = set1.symmetric_difference(set2)
print(f"symmetric_difference removes all common values. Thus:\n\t{set1
=}\nsymmetric_difference\n\t{set2 =}\ngives\n\t{set3 =}\n")
#
#   symmetric_difference_update() changes the calling set removing all common
elements):
#
set1 = {"apple", "banana", "cherry"}
set2 = {"samsung", "redmi", "apple"}

set1.symmetric_difference_update(set2)
print(f"symmetric_difference_update removes all common values and updates the
calling set.\n\t{set1 =}\n")

```

difference()

Method returns a new set with values that are in the calling set only

The method can work with other iterables as well

The method can take multiple arguments

Shorthand is the - operator, but works only with sets

`difference_update()`

Changes the calling set leaving values ONLY present in the calling set

`symmetric_difference()`

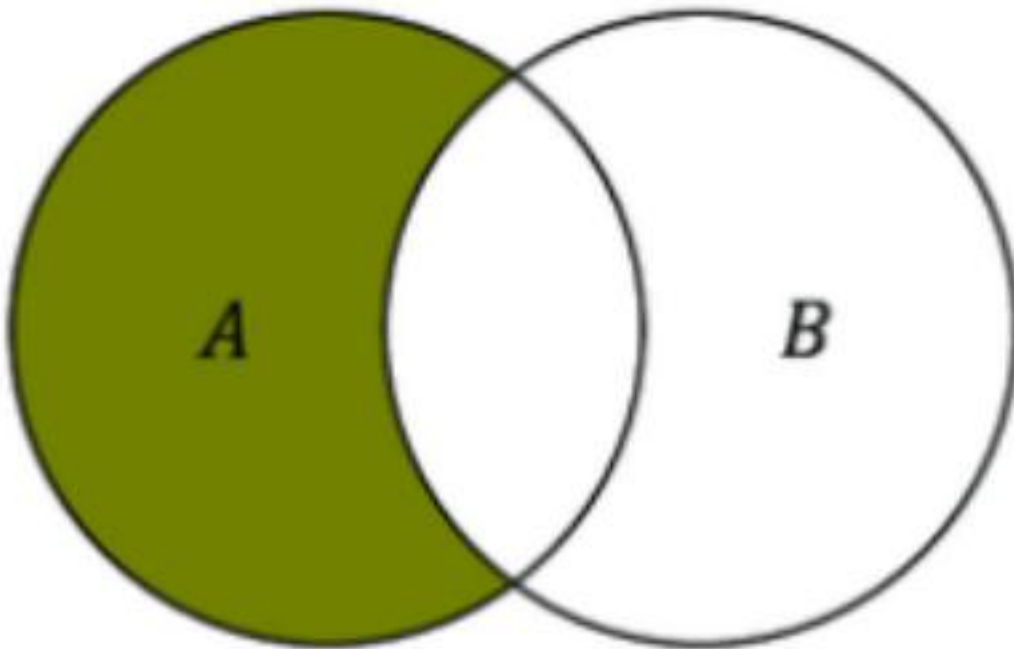
Returns a new set with all common values removed

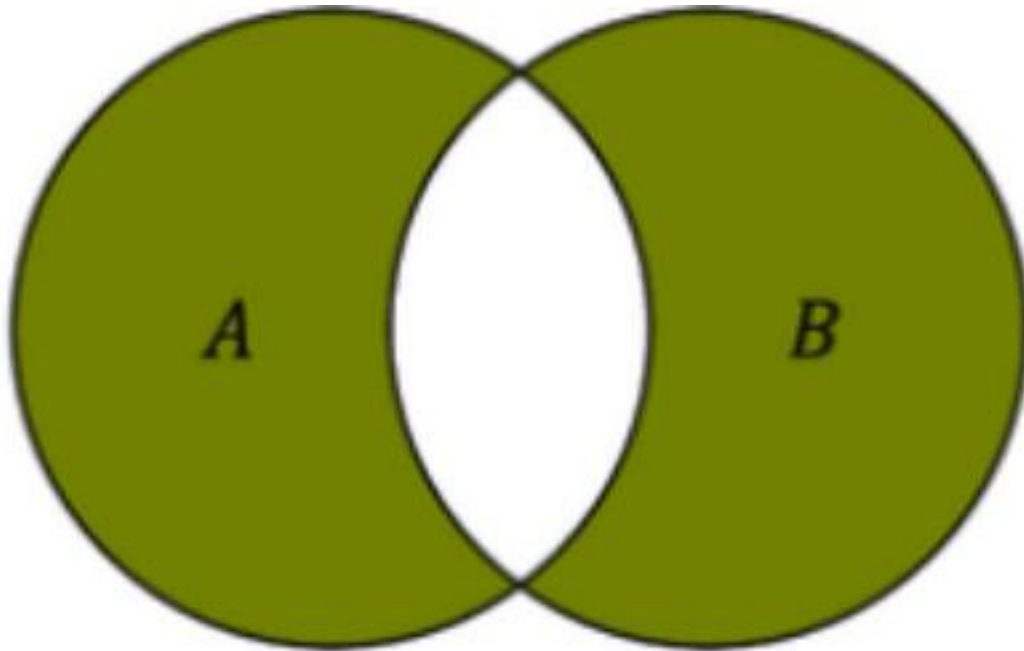
Shorthand is the ^ operator

`symmetric_difference_update()`

Changes the calling set, removing all common values

The `difference()` method will return a new set that will contain only the items from the first set that are not present in the other set.





Python Code: 8_sets_methods.py

```
#
#   Miscellaneous set methods
#
set1 = {1, 2, 3, 4}
set2 = set1
set1.add(100)
print(f"A shallow copy does not work. {set1=}, {set2 =}\n")
set3 = set1.copy()
set1.add(200)
print(f"The copy() method works and returns a new set:\n\t{set1=}\n\t{set3
=}\n")
#
#   The set constructor can also be used to create a copy
#
set3a = set(set1)
set1.add(300)
print(f"The set() constructor works and returns a new set:\n\t{set1=}\n\t{set3a
=}\n")
#
#   The following methods return boolean values
#
#   isdisjoint()
#
set1 = {1,2,3}
set2 = {3,4,5}
set3 = {4,5,6}
print(f"isdisjoint() returns True if sets do not have any common
values:\n\t{set1 =}\n\t{set2 =}\n\t{set1.isdisjoint(set2) =}\n")
print(f"\t{set3 =}\n\t{set1.isdisjoint(set3) =}\n")
#
#   issubset()
```

```
#
set1 = {1,2,3}
set2 = {1,2,3,4,5}
print(f"issubset() returns True if the calling set is a subset of the
argument:\n\t{set1 =}\n\t{set2 =}\n\t{set1.issubset(set2) =}")
print(f"\t{set2.issubset(set1) =}\n")
#
#    issuperset()
#
set1 = {1,2,3}
set2 = {1,2,3,4,5}
print(f"issuperset() returns True if the calling set is a superset of the
argument:\n\t{set1 =}\n\t{set2 =}\n\t{set1.issuperset(set2) =}")
print(f"\t{set2.issuperset(set1) =}\n")
```

Copy()	-	returns a copy of a set (deep copy)
Isdisjoint()	-	whether two sets have common elements
Issubset()	-	whether one set is a subset of the other
Issuperset()	-	whether one set is a superset of the other
Frozenset()	-	constructor for a frozen set

Examples: Use of sets
 The power of Python!

Python Code: 9_sets_example.py

```
#
#    Some examples of set usage
#
Students = ["Abid","Bashir","Chaudhry","Dogar","Elahi","Fazal","Ghafoor"]
Mess_members = ["Bashir","Chaudhry","Dogar","Hafeez"]
Gym_members = ["Abid","Bashir","Chaudhry","Ijaz"]
#
#    Find all students who are Mess_members and Gym_members
#    (Begins by converting Student list to a set for calling set methods)
#
Mess_and_Gym_Students = set(Students).intersection(Mess_members, Gym_members)
print(f"Students who are Mess and Gym members:\n{Mess_and_Gym_Students}\n")
#
#    Find students who are not Mess_members nor Gym_members
#
Not_Mess_Gym = set(Students).difference(Mess_members, Gym_members)
print(f"Students who are neither Mess nor Gym members:\n{Not_Mess_Gym}\n")
#
#    Find students who are Mess_members but not Gym_members
#
Mess_not_Gym = set(Students).intersection(Mess_members).difference(Gym_members)
print(f"Students who are Mess members but not Gym members:\n{Mess_not_Gym}\n")
```

Python Code: 10_sets_power_of_Python-1.py

```
str1 = """To be, or not to be, that is the question:
Whether 'tis nobler in the mind to suffer
```

```

The slings and arrows of outrageous fortune,
Or to take arms against a sea of troubles
And by opposing end them. To die – to sleep,
No more; and by a sleep to say we end
The heart-ache and the thousand natural shocks
That flesh is heir to : 'tis a consummation
Devoutly to be wish'd. To die, to sleep;"""
#
# Find the distinct words in the above paragraph ignoring upper/lower case
#
# Convert string to lower case
#
str2 = str1.lower()
#
# Now split the string into a list of words
#
list1 = str2.split()
#
# Finally convert the list into a set which will drop all duplicate words
#
distinctwords = set(list1)
print(f"The distinct words in the paragraph are:\n{distinctwords}")
#
# If a sorted list is needed, a few extra steps are required-> convert to
list, sort list, then print
# mylist = list(set(str1.lower().split()))
# mylist.sort()
# print(mylist)

```

Python Code: 10_sets_power_of_Python-2.py

```

str1 = """To be, or not to be, that is the question:
Whether 'tis nobler in the mind to suffer
The slings and arrows of outrageous fortune,
Or to take arms against a sea of troubles
And by opposing end them. To die – to sleep,
No more; and by a sleep to say we end
The heart-ache and the thousand natural shocks
That flesh is heir to : 'tis a consummation
Devoutly to be wish'd. To die, to sleep;"""
#
# Find the distinct words in the above paragraph ignoring upper/lower case
#
print(f"The distinct words in the paragraph are:\n{set(str1.lower().split())}")
list2 = list(set(str1.lower().split()))
list2.sort()
print(list2)

```

Python Code: 0_dict.py

```

#
# Creating a dictionary by giving key:value pairs in {}
#
first_dict = {

```

```

    "play": "Hamlet",
    "author": "Shakespeare",
    "year": 1600,
    "soliloquy": "To be or not to be"
}
print(f"{first_dict = }\n")          # Prints as a dictionary with {}
and comma separated key:value pairs
#
# Creating a dictionary by using the dict constructor
#
second_dict=dict(key1 = 1, key2 = "You", key3 = "Me")
print(f"{second_dict = }\n")
#
# Creating an empty dictionary by using {} or the dict constructor
#
third_dict = {}
print(f"Empty dictionary created using empty braces, {third_dict = }\n")
fourth_dict = dict()
print(f"Empty dictionary created using dict(), {fourth_dict = }\n")
#
# Using the standard Python functions
#
print(f"{type(first_dict) = }, {len(first_dict) = }\n")

```

Dictionaries

One of four Python data types used to store collections

(Others are sets, lists, tuples)

Used to store data values in key:value pairs

Dictionaries are ordered*, changeable and do not allow duplicates

Ordered – Items have a defined order that does not change

Changeable – Items can be added or deleted, and values can be changed as well

No duplicates – keys have to be unique; duplicate keys not allowed although values may be repeated

Created by using

{key:value,...} or

dict(key1 = value, key2 = value2...) or

dict() / {} for an empty dictionary or

Can contain any type(s) of data – mixed items allowed

Ordered and indexed by keys (not positions), items are changeable

No duplicate keys allowed!

(Python functions type() and len() apply!)

Accessing dictionary items

Items can be accessed using their key:

Value = dictionary[key]

Or using the get() method:

Value = dictionary.get(key)

All keys can be obtained as a special list using the method:

dictionary.keys()

All values can be obtained as a special list using the method:

dictionary.values()

All key:value pairs can be obtained using the method:

Dictionary.items()

Finding dictionary items

Items can be found using the membership operator

Works on the keys of the dictionary

Be careful with keys having mixed data types!

Preferable to have keys of the same data type (like a dictionary!)

Changing dictionary items

An item can be changed by referring to its key

An item can be added by providing a new key

The update method allows an iterable to be used

Iterable must have the key,value pairs

The update method will change the value of an existing key

Existing keys will have values updated, new ones will be added

Removing items

Items can be removed using

Pop(key,default) method with key provided

Popitem() removes the last item added

Del dictionary[key] removes the item with the specified key

As with lists, sets and tuples,

Clear – deletes all items in a dictionary

Del dictionary – removes the dictionary from memory

Looping through dictionaries

For loops can be used with membership

With dictionary name only – loop variable is dictionary key

With dictionary.keys() – loop variable is dictionary keys

With dictionary.values() – loop variable is dictionary values

With dictionary.items() – loop variables are dictionary (key, value) tuples

Copying Dictionaries

A shallow copy does not work:

Dict2 = dict1 simply creates a reference

Two ways for create a deep copy:

The copy() method

The dict() constructor

Nested Dictionaries

Dictionaries can contain other dictionaries and so on

Known as “nested” dictionaries

Useful when organizing data of similar type e.g. employees

Items are accessed using stacked key indexing – [][]

Other Dictionary methods

Fromkeys(list/tuple, value)

Method of dict() class

Creates a new dictionary with keys specified in the list/tuple

If a value is specified, the same value is given to all keys

Setdefault(key, value)

Returns the value of the item with the specified key (value is ignored)

If key does not exist, it is created and the value assigned

If the value is not provided, the key is created with None as the value

Other Dictionary concepts

Dictionary.keys()

Dictionary.values()

Dictionary.items()

Are all views into the dictionary. Once created, they are automatically updated to incorporate any changes to keys or values and reflect the current state of the dictionary

Python Code: 1_dict_access.py

```
#
# Create a dictionary
#
first_dict = {
    "play": "Hamlet",
    "author": "Shakespeare",
    "year": 1600,
    "soliloquy": "To be or not to be",
}
#
# Access an item using a key
#
print(f"The value of the key \"play\" is {first_dict[\"play\"]}\n")
#
# Using the get() method:
#
print(f"The value of the key \"play\", using the get() method is:
{first_dict.get(\"play\")}\n")
#
# All keys can be obtained as a dict_keys list object using the keys() method
#
print(f"The keys of the dictionary are: {first_dict.keys()}\n")
#
# All values can be obtained as a dict_values list object using the values()
method
#
print(f"The values of the dictionary are: {first_dict.values()}\n")
#
# All key:value pairs can be obtained as a list of tuples using the items()
method
#
print(f"The key:value pairs in the dictionary are: \n{first_dict.items()}\n")
```

Items can be accessed using their key:

Value = dictionary[key]

Or using the get() method:

Value = dictionary.get(key)

All keys can be obtained as a special list using the method:

dictionary.keys()

All values can be obtained as a special list using the method:

dictionary.values()

All key:value pairs can be obtained using the method:

Dictionary.items()

Python Code: 2_dict_find.py

```
#
# Finding an item using a key with the membership operator
#
first_dict = {
    "play": "Hamlet",
    "author": "Shakespeare",
    "year": 1600,
    "soliloquy": "To be or not to be",
    1 : 1000
}
find_key = input("Which key would you like to find? ")
if find_key in first_dict:
    print(f"\nFound! The value of the key \"{find_key}\" is:
{first_dict[find_key]}\n")
else:

    # It is preferable to have keys of the same data type otherwise extra
    # programming is needed

    if find_key.isnumeric():
        find_key = int(find_key)
    if find_key in first_dict:
        print(f"\nFound! The value of the key \"{find_key}\" is:
{first_dict[find_key]}\n")
    else:
        print(f"\nKey \"{find_key}\" does not exist in the dictionary\n")
```

Items can be found using the membership operator

Works on the keys of the dictionary

Be careful with keys having mixed data types!

Preferable to have keys of the same data type (like a dictionary!)

Python Code: 3_dict_change.py

```
#
# Changing values and items in dictionaries
#
my_dict = {
    "play": "Hamlet",
    "author": "Shakespeare",
    "year": 1600,
    "soliloquy": "To be or not to be",
}
#
# Change a value by referring to key
#
print(f"Original Dictionary: \n{my_dict}\n")
my_dict["year"] = 1601
print(f"Dictionary with year changed by referring to key: \n{my_dict}\n")
#
```

```

# Adding an item by providing a new key
#
my_dict["publisher"] = "Nicholas Ling"
print(f"Dictionary with publisher added by giving a new key: \n{my_dict}\n")
#
# Adding items using the update method
# First, update with a dictionary
#
dict_upd={"act":"3", "scene":"1" }
my_dict.update(dict_upd)
print(f"Dictionary after update with another dictionary: \n{my_dict}\n")
#
# Now, update with a list of lists
#
list_upd=[["speaker","Hamlet"], ["words", 100]]
my_dict.update(list_upd)
print(f"After update with a list of lists: \n{my_dict}\n")
#
# Update with a tuple of tuples
#
tup_upd=(("audience","Alone"), ("words", 250))
my_dict.update(tup_upd)
print(f"After update with a tuple of tuples: \n{my_dict}\n")
#
# We can update with a list of tuples
#
mix_upd = [("tuple1","T"), ("tuple2","L") , ("tuple3","D")]
my_dict.update(mix_upd)
print(f"After update with list of tuples: \n{my_dict}\n")

```

An item can be changed by referring to its key

An item can be added by providing a new key

The update method allows an iterable to be used

Iterable must have the key,value pairs

The update method will change the value of an existing key

Existing keys will have values updated, new ones will be added

Python Code: 4_dict_del.py

```

my_dict = {'play': 'Hamlet', 'author': 'Shakespeare', 'year': 1601,
           'soliloquy': 'To be or not to be', 'publisher': 'Nicholas Ling',
           'act': '3',
           'scene': '1', 'speaker': 'Hamlet', 'words': 250, 'audience': 'Alone'}
#
# Items can be deleted by using the pop method and specifying the key
#
print(f"Original Dictionary: \n{my_dict}\n")
pop_value = my_dict.pop("audience","Not found")
print(f"The value popped for the key \"audience\" is \n\t{pop_value} \nand the

```

```

remaining Dictionary is: \n{my_dict}\n")
#
# The last item can be deleted by using the popitem() method
#
pop_value = my_dict.popitem()
print(f"The item popped is \n\t{pop_value} \nand the remaining Dictionary is:
\n{my_dict}\n")
#
# Items can be deleted by using the del function and specifying the dictionary
name and key
#
del my_dict["scene"]
print(f"Item deleted using del fn on my_dict with the key \"scene\" leaving the
remaining Dictionary as: \n{my_dict}\n")
#
# The entire dictionary can be cleared
#
my_dict.clear()
print(f"Dictionary cleared: \n{my_dict} = {}\n")
#
# Finally, the entire dictionary is removed from memory
#
del my_dict
# print(my_dict)          # This will now give an error

```

Items can be removed using

Pop(key,default) method with key provided

Popitem() removes the last item added

Del dictionary[key] removes the item with the specified key

As with lists, sets and tuples,

Clear – deletes all items in a dictionary

Del dictionary – removes the dictionary from memory

Python Code: 5_dict_loops.py

```

#
# Looping through dictionaries
#
my_dict = {'play': 'Hamlet', 'author': 'Shakespeare', 'year': 1601,
          'soliloquy': 'To be or not to be', 'publisher': 'Nicholas Ling',
          'act': '3',
          'scene': '1', 'speaker': 'Hamlet', 'words': 250, 'audience': 'Alone'}
print(f"The Dictionary is: \n{my_dict}")
#
# Looping using just the dictionary name
#
print("\nLooping using just the dictionary name, the loop var is the key:")
for loop_var in my_dict:
    print(loop_var)          # Prints the key
#
# Looping using dictionary.keys()

```

```

#
print("\nLooping using dictionary.keys(), the loop var is the key:")
for loop_var in my_dict.keys():
    print(loop_var)          # Prints the key
#
#   Looping using dictionary.values()
#
print("\nLooping using dictionary.values(), the loop var is the value:")
for loop_var in my_dict.values():
    print(loop_var)          # Prints the values
#
#   Looping using dictionary.items()
#
print("\nLooping using dictionary.items(), the loop vars are (key, value)
tuples:")
for loop_var in my_dict.items():
    print(loop_var)          # Prints the keys and values. Can index the tuple
if needed

```

For loops can be used with membership

With dictionary name only – loop variable is dictionary key

With dictionary.keys() – loop variable is dictionary keys

With dictionary.values() – loop variable is dictionary values

With dictionary.items() – loop variables are dictionary (key, value) tuples

Python Code: 6_dict_copy.py

```

#
# Creating our work dictionary
#
first_dict = {
    "play": "Hamlet",
    "author": "Shakespeare",
    "year": 1600,
    "soliloquy": "To be or not to be"
}
#
#   Trying to make a copy using a simple assignment
#
second_dict = first_dict
print(f"Copy created using simple assignmnet:\n{first_dict = }\n{second_dict
= }\n")
#
#   Changing a value in first_dict
#
first_dict["play"] = "Macbeth"
print(f"Value of 'play' changed in first_dict changes second_dict as
well:\n{first_dict = }\n{second_dict = }\n")
#
# Copy created using the copy() method
#
second_dict = first_dict.copy()
print(f"Copy created using the copy() method:\n{first_dict = }\n{second_dict

```

```

= }\n")
#
#   Changing a value in first_dict
#
first_dict["play"] = "Hamlet"
print(f"Value of 'play' changed in first_dict does NOT change
second_dict:\n{first_dict = }\n{second_dict = }\n")
#
# Copy created using the dict() constructor
#
second_dict = dict(first_dict)
print(f"Copy created using the dict() constructor:\n{first_dict
= }\n{second_dict = }\n")
#
#   Changing a value in first_dict
#
first_dict["play"] = "Othello"
print(f"Value of 'play' changed in first_dict does NOT change
second_dict:\n{first_dict = }\n{second_dict = }\n")

```

A shallow copy does not work:

Dict2 = dict1 simply creates a reference

Two ways for create a deep copy:

The copy() method

The dict() constructor

Python Code: 7_dict_nested.py

```

#
#   Create multiple dictionaries - students
#
student1 = {"name": "Ahmad", "age": 19, "grade": "A"}
student2 = {"name": "Bilal", "age": 22, "grade": "B+"}
student3 = {"name": "Durrani", "age": 20, "grade": "B"}
#
#   Creating a nested dictionary, using key:value pairs where each value is
itself a dictionary
#
my_class = {"std1": student1, "std2": student2, "std3": student3}
print(f"Nested dictionaries: \n{my_class = }\n")
#
#   Accessing items from nested dictionaries. Use key names for outer and inner
dicts
#
print(f"{my_class['std1']['name']} = }\n")
#
#   Hence, looping through the keys would be like:
#
for student in my_class:
    print(student)
    for key2 in my_class[student]:

```

```

        print(f"\t{key2} : {my_class[student][key2]}")        # The second
variable prints the value of the nested dict item
#
#   Dictionaries can be nested further. e.g three layers
#
neighborhood = {
    "house1":{
        "name":{"first":"Abdul","middle":"H","last":"Mateen"},
        "address":{"housetno":"1-A","street":"Green
Street","mohalla":"Gulberg","city":"Lahore"}
    },
    "house2":{
        "name":{"first":"Sheikh","middle":"Idrees","last":"Ahmad"},
        "address":{"housetno":"2","street":"Blue Street","mohalla":"Gulberg-
II","city":"Lahore"}
    }
}
print(neighborhood["house1"]["address"]["mohalla"])
#
#   Triple nested loop
#
for nbrhood in neighborhood:
    print(nbrhood)
    for key2 in neighborhood[nbrhood]:
        print(f"\t{key2} :")
        for key3 in neighborhood[nbrhood][key2]:
            print(f"\t\t{key3} : {neighborhood[nbrhood][key2][key3]}")
#

```

Dictionaries can contain other dictionaries and so on

Known as “nested” dictionaries

Useful when organizing data of similar type e.g. employees

Items are accessed using stacked key indexing – [][]

Python Code: 8_dict_methods.py

```

#
#   Creating a new dictionary from a list of keys, values will be None
#
# car = {
#     "brand": "Ford",
#     "model": "Mustang",
#     "year": 1964
# }
key_list = ["brand", "model", "year"]
my_dict=dict.fromkeys(key_list)
print(f"Dictionary created from a key list without values:\n{my_dict}\n")
#
#   Creating a new dictionary from a list of keys with the same value for every

```

```

key
#
my_dict=dict.fromkeys(key_list,"Ford")
print(f"Dictionary created from a key list with a value provided:\n{my_dict}\n")
#
#   Let us set the value to something proper
#
my_dict["model"]="Mustang"
my_dict["year"]=1964
print(f"Dictionary with values updated:\n{my_dict}\n")
# -----
-----
#   setdefault(key,value)
#   Returns the value for the key provided
#
val = my_dict.setdefault("model")
print(f"The setdefault method returns the value associated with 'model':{val}")
#
#   If the key exists, the existing value is returned and the provided value
ignored. No other change
#
val = my_dict.setdefault("model","Fairlane")
print(f"The setdefault method returns the value associated with 'model',
ignoring the value in the calling statement: {val}\n")
#
#   If the key does not exist, it is created and a value (if provided) is
assigned
#
val = my_dict.setdefault("cartype", "saloon")
print(f"If the key does not exist, it is created and a value (if provided) is
assigned:\nThus my_dict.setdefault('cartype', 'saloon') changes the dictionary
to:\n{my_dict}")
print(f"and the value returned is: {val}\n")
#
#   and add another key and value
#
color = my_dict.setdefault("color", "white")
print(f"The updated dictionary is: \n{my_dict}\n")

```

Fromkeys(list/tuple, value)

Method of dict() class

Creates a new dictionary with keys specified in the list/tuple

If a value is specified, the same value is given to all keys

Setdefault(key, value)

Returns the value of the item with the specified key (value is ignored)

If key does not exist, it is created and the value assigned

If the value is not provided, the key is created with None as the value

Python Code: 9_dict_views.py

```

#
#   Let us start with our car dictionary

```

```

#
my_car = {'brand': 'Ford', 'model': 'Mustang', 'year': 1964, 'cartype':
'saloon', 'color': 'white'}
#
#   Obtain the keys, values and items from this dictionary
#
car_keys = my_car.keys()
car_values = my_car.values()
car_items = my_car.items()
#
#   Print the type and values of these lists
#
print(f"The type of car_keys is: {type(car_keys)}\nand the list is:
\n{car_keys}\n")
print(f"The type of car_values is: {type(car_values)}\nand the list is:
\n{car_values}\n")
print(f"The type of car_items is: {type(car_items)}\nand the list is:
\n{car_items}\n")
#
#   Now update the dictionary
#   Update two keys
#
my_car['brand'] = 'Mercedes'
my_car['model'] = 'E class'
print(f"Two values updated. The car_values is a view and shows updated
values:\n{car_values}\n")
#
#   And add a new key
#
my_car['interior'] = "leather"
print(f"A new key added. The car_keys is a view and shows updated
keys:\n{car_keys}\n")
#
#   And finally, car_items is a view of the dictionary items and is also
updated:
print(f"The car_items is also a view and stays updated:\n{car_items}\n")

```

Dictionary.keys()

Dictionary.values()

Dictionary.items()

Are all views into the dictionary. Once created, they are automatically updated to incorporate any changes to keys or values and reflect the current state of the dictionary

Python Code: 0_file_open.py

```

#
#   Opening a file
#
fh = open("sample.txt")           # Default is Text file in Read mode.
File must exist
str = fh.read()                   # Reads entire file contents
print(str)                        # Prints the file contents

```

```

fh.close()                # Must always close a file after using it.
#
#   Opening a file not in the current directory
#   Need to specify the complete or relative path name.
#   Remember to escape the backslash characters
#
print("\nOpening a file in a different folder using a relative path name:\n")
fh = open("../session 9\\0_dict.py")
str = fh.read()           # Reads entire file contents
print(str)                # Prints the file contents
fh.close()                # Must always close a file after using it.

```

Files in Python

Files are broadly categorized as

- text files
- binary files

File Handling in Python

Manages data flow between the program and the file system

Primary functions performed on a file are

- Opening
- Reading
- Writing
- Closing

File operations

Opening

File_handle = open("name of file", mode)

where the name of the file is a string containing

- simple name if the file is in the same directory as the Python program

- full path name if file is somewhere else

and mode specifies how the file is to be used

Reading

File_handle.read() – reads the entire file

Must close a file after it has been used

File_handle.close()

Python Code: 0_dict.py

```
#
# Creating a dictionary by giving key:value pairs in {}
#
first_dict = {
    "play": "Hamlet",
    "author": "Shakespeare",
    "year": 1600,
    "soliloquy": "To be or not to be"
}
print(f"{first_dict = }\n")          # Prints as a dictionary with {}
and comma separated key:value pairs
#
# Creating a dictionary by using the dict constructor
#
second_dict=dict(key1 = 1, key2 = "You", key3 = "Me")
print(f"{second_dict = }\n")
#
# Creating an empty dictionary by using {} or the dict constructor
#
third_dict = {}
print(f"Empty dictionary created using empty braces, {third_dict = }\n")
fourth_dict = dict()
print(f"Empty dictionary created using dict(), {fourth_dict = }\n")
#
# Using the standard Python functions
#
print(f"{type(first_dict) = }, {len(first_dict) = }\n")
```

Python Code: 1_file_read.py

```
#
# read() reads the entire file at a go
#
fh = open("sample.txt")
contents = fh.read()
print(f"The read() functions reads the entire contents of the file, giving:\n")
print(contents, "\n")
fh.close()
#
# readline() reads the file one line at a time, returning a string including
the \n character
#
fh = open("sample.txt")
line = fh.readline()
print(f"readline() returns the next line: \n{line}")
line = fh.readline()
print(line)
line = fh.readline()
print(line)
print("*** Notice the extra space after the lines above?***\n\n")
fh.close()
#
```

```

# readlines() reads the entire file, returning a list of strings, each
including the \n character
#
fh = open("sample.txt")
list_lines = fh.readlines()
print(f"Readlines() gives a list of strings:\n{list_lines}")
fh.close()
#
# read(n) reads the file n characters at a time
#
print("\nReading the file 10 characters at a time.\n")
fh = open("sample.txt")
chunk = fh.read(10)
while len(chunk) > 0:
    print(chunk, end='')
    chunk = fh.read(10)
fh.close()
#
# looping through a file using the membership operator, line by line
#
print("\n\nLooping through the file line by line using the membership
operator\n")
fh = open("sample.txt")
for line in fh:
    print(line, end='')
fh.close()

```

File opening modes

- "r" - open for reading; assumes the file is plain text
- "w" - opens a file for writing; will create file if it does not exist
 - will truncate it to 0 bytes if it exists! – Be careful
- "a" - append to a file; continues adding content to an existing file
 - will create file if it does not exist
- "x" - exclusive create file; gives error if file exists
 - can write to the file but cannot read it

File operations - reading

Reading file contents

- Read() - returns entire contents of file as a string
- Readline() - returns one line of a file including \n character
- Readlines() - returns all lines of a file as a list of strings
 - each one including \n character
- Read(n) - read (max) n characters/bytes at a time

Membership operator - loops through file line by line

Python Code: 2_file_append_write.py

```
#
#   Appending text to an existing file
#
fh = open("sample2.txt","a")
fh.write("This text has been added to the file\n")
fh.close()
# #
# #   If we open a file in w mode, the original contents are lost!
# #
# fh = open("sample2.txt","w")
# fh.write("The original contents are gone and replaced with this line!\n")
# fh.close()
# #
# #   If we open a file in x mode, a new file will be created. If it exists an
# #   error will occur
# #
# fh = open("sample3.txt","x")
# fh.write("This is a new file created with x mode!\n")
# fh.close()
```

Python Code: 2a_file_writelines.py

```
#
#   Writing a list to a file - use writelines()
#
list_lines=["This is the first line\n","This is the second line\n","This is the
third line\n", "Finally, this is the fourth line\n"]
fh = open("new.txt","w")
fh.writelines(list_lines)
fh.close
```

File operations – open for writing

- | | |
|----------|--|
| Mode “a” | - appends to an existing file
- will create file if it does not exist |
| Mode “w” | - will create a new file if it does not exist
- will overwrite contents if file exists! |
| Mode “x” | - will create a new file if it does not exist
- will thrown an error of file exists – safe! |

Python Code: 2a_file_writelines.py

```
#
#   Writing a list to a file - use writelines()
#
list_lines=["This is the first line\n","This is the second line\n","This is the
third line\n", "Finally, this is the fourth line\n"]
fh = open("new.txt","w")
```

```
fh.writelines(list_lines)
fh.close
```

File operations - writing

Writing to a file

`write()` - writes content to a file

`writelines()` - writes the items of a list to the file

- list items must be strings
- should include `\n` for proper line breaks

Python Code: 3_file_copy_txt.py

```
#
# Copying a text file
#
# Ask user which file to copy
# Target will be called source_copy.txt
#
source = input("Which file do you want to copy?")
destination = source[0:source.find(".")]+"_copy.txt"
fi = open(source, "r")
fo = open(destination, "w")
fo.write("This file has been created by reading/writing the entire file at one
go\n\n")
content = fi.read()
fo.write(content)
fi.close()
fo.close()

#
# Copying a text file line by line (useful for large files)
#
destination = source[0:source.find(".")]+"_copy2.txt"
fi = open(source, "r")
fo = open(destination, "w")
fo.write("This file has been created by reading/writing the file line by line
using a loop\n\n")
for line in fi:
    fo.write(line)
fi.close()
fo.close()
```

File opening combination modes

`rb` - read binary file (in bytes)

`r+` - read and write mode

`rb+` - read and write binary file

`wb` - write binary file

`w+` - write and read; truncates file if it exists

wb+ - write and read binary file; truncates file if it exists
ab - append in binary mode
a+ - append and read
ab+ - append and read binary
xb - exclusive creation binary; error if file exists
x+ - exclusive creation and reading; error if file exists
xb+ - exclusive creation and reading binary; error if file exists

Example – copying a file

Copying a text file

Entire file

Line by line

(Can be done chunk by chunk also!)

Copying a binary file

Entire file

Chunk by chunk

Python Code: 4_file_copy_bin.py

```
#
#   Copying a binary file. Both files must be opened with "b" added to mode
#
#   Ask user which file to copy
#   Target will be called source_copy.ext
#
source = input("Which binary file do you want to copy?")
destination =
source[0:source.find(".")]+"_copy"+source[source.find("."):len(source)]
fi = open(source, "rb")
fo = open(destination, "wb")
content = fi.read()
fo.write(content)
fi.close()
fo.close()
# #
# #   Copying a binary file chunk by chunk (useful for large files)
# #
destination =
source[0:source.find(".")]+"_copy2"+source[source.find("."):len(source)]
fi = open(source, "rb")
fo = open(destination, "wb")
chunk_size=100
chunk = fi.read(chunk_size)
while len(chunk) > 0:
    fo.write(chunk)
    chunk = fi.read(chunk_size)
fi.close()
fo.close()
```

Copying a text file

Entire file

Line by line

(Can be done chunk by chunk also!)

Copying a binary file

Entire file

Chunk by chunk

Python Code: 5_file_posn.py

```
#
#   When reading/writing to a file, one can find the position where
reading/writing will take place
#
fi = open("sample.txt", "r")
fo = open("messed_up.txt", "w")
chunk_size=75
chunk = fi.read(chunk_size)
while len(chunk) > 0 and fi.tell()<200:
    print("Positions in the input and output files are: ",fi.tell(),fo.tell())
    fo.write(chunk)
    chunk = fi.read(chunk_size)
#
#   Now position back to the top of sample.txt and write some junk into the
file
#   Also position backwards in output file. Moving forward will introduce NULLs
into the file
#
fo.write("\n=====\\n")
fi.seek(0)
print("Output file is at position:",fo.tell())
print("re-positioning output file to:",fo.tell()-10)
fo.seek(fo.tell()-10)
chunk = fi.read(chunk_size)
while len(chunk) > 0:
    print("Positions in the input and output files are: ",fi.tell(),fo.tell())
    fo.write(chunk)
    chunk = fi.read(chunk_size)
fi.close()
fo.close()
```

Position in a file

Seek()

Tell()

Example

Positioning in input and output files

Example

Search and replace text in a file

Translate according to a dictionary

Using context

File handling requires precise management of resources

Opened files must be closed

Exceptions may happen when reading or writing files

Clean technique, use context:

with open(file,mode) as file_handle:

 file.action() (read, write etc.)

No need to close the file. Exiting the block will close it.

Even if exceptions happen, file will be properly closed.

Python Code: 6_file_srch_repl.py

```
#
#   Replacing text in a file
#
#   Ask user for filename and search and replace texts
#
source = input("Which file do you want to process?")
srch_string = input("String to search for: ")
repl_string = input("Replace with: ")
fi = open(source, "r+")
#
#   Read the entire file, process, reposition, and write back
#   Works if file is small and len(srch_str) >= (repl_str) otherwise we get
garbage at the end
#   Alternative would be to close the file and reopen in "w" mode thereby
overwriting it.
#
contents = fi.read()
new_contents = contents.replace(srch_string, repl_string)
fi.seek(0)
fi.write(new_contents)
fi.close()
#
#   Let us try a line by line approach. It ONLY works if the search and
#   replace strings are the same length. Otherwise we get all kind of problems
#
source = input("Which file do you want to process?")
srch_string = input("String to search for: ")
repl_string = input("Replace with: ")
if len(srch_string) == len(repl_string):
    fi = open(source, "r+")
    #
    #   Read the file line by line, process, write the line
    #
    position = fi.tell()
    contents = fi.readline()
    while len(contents) > 0:
```

```

        new_contents = contents.replace(srch_string, repl_string)
        #
        # Now position file at original location
        #
        fi.seek(position)
        fi.write(new_contents)
        position = fi.tell()
        contents = fi.readline()
    fi.close()
else:
    print("Search and replace strings are different lengths. Aborting!")
#
# Finally, for very large files, or for search and replace strings of
# different lengths, it is easier to read from one file. do the replacements
and write to another.
#
source = input("Which file do you want to process?")
srch_string = input("String to search for: ")
repl_string = input("Replace with: ")
fi = open(source, "r+")
fo = open(source+".new", "w")
#
# Read the file line by line, process, write the line to a second file
# We can even loop through the input file with the membership operator
#
for line in fi:
    new_contents = line.replace(srch_string, repl_string)
    fo.write(new_contents)
fi.close()
fo.close()

```

Seek()

Tell()

Example

Positioning in input and output files

Example

Search and replace text in a file

Translate according to a dictionary

Python Code: 7_file_dict_trans.py

```

#
# Translating text in a file from a given dictionary
trans_dict = {"me":"myself", "thee":"thou", "Dagger":"Knife"}
#
# Ask user for filename and search and replace texts
#
source = input("Which file do you want to process?")
fi = open(source, "r+")
#

```

```
# Read the entire file, process, and write back
# Works if file is small. Does have problems with key and value lengths
# Alternative would be to close the file and reopen in "w" mode thereby
overwriting it.
#
contents = fi.read()
for key, value in trans_dict.items():
    contents = contents.replace(key, value)
fi.seek(0)
fi.write(contents)
fi.close()
```

Seek()

Tell()

Example

Positioning in input and output files

Example

Search and replace text in a file

Translate according to a dictionary

Python Code: 8_file_context.py

```
#
# Using context to ensure proper resource cleanup
#
with open("sample.txt") as fh:
    content = fh.read()
with open("sample_context.txt", "w") as fh:
    fh.write(content)
print(content)
#
# Or using one context for multiple files and changing all text to upper case.
#
with open('sample.txt', 'r') as input_file, open('output.txt', 'w') as
output_file:
    content = input_file.read()
    output_file.write(content.upper())
#
# At this point, both files have been properly closed
#
print("Job completed")
```

File handling requires precise management of resources

Opened files must be closed

Exceptions may happen when reading or writing files

Clean technique, use context:

```
with open(file, mode) as file_handle:
    file.action() (read, write etc.)
```

No need to close the file. Exiting the block will close it.
Even if exceptions happen, file will be properly closed.

Python Code: 0_0_fn_test.py

```
def fn(n):
    if n==1:
        return 1
    elif n==2:
        return 1,2
    else:
        return 1,2,3

print(fn(10))
```

Python Code: 0_fn_recap1.py

```
#
#   Function with positional parms; calling args must be in the same order
#
def func_posn(name, age, town):
    print(f"My name is {name}, I am {age} years old and I live in {town}")
#
#   Calling the above
#
func_posn("Javed", 20, "Rawalpindi")

#
#   Function with positional parms and default value; calling args must be in
the same order but default may be omitted
#
def func_posn_def(name, age, town="Lahore"):
    print(f"My name is {name}, I am {age} years old and I live in {town}")

func_posn_def("Asghar", 25)    # Remove Rawalpindi - defaults to Lahore
#
#   Call with keywords
#
func_posn_def(age = 30, name = "Suleman")    # Remove Okara - defaults to
Lahore, order not important
```

Functions – Recap

Functions may be called with:

Positional arguments

func(a,b,c)

Default arguments

func(a,b,c=10)

Keyword arguments

func(b=10, a=20, k=30)

Functions – Definition vs Call

Definition:

```
def func(x, y, z, a=10, b=20):
```

Call:

```
func(a1, b1, c1, b=5, a=3)
```

positional

default

keyword

Functions – Revisited

Number of arguments used to call must match the number of parameters defined in the function unless the function defines default parameters

Arguments must be in the correct order

If key=value syntax is used in calling, then order does not matter. However, key MUST be defined in the function

Python Code: 1_fn_var_args.py

```
#
#   Defining a function to receive a variable number of arguments
#
def fn(*args):
    print(f"\nThis function receives a {type(args)} consisting of the values:
{args}, which are {len(args)} in number\n")

fn("Hello", "World")

def add(*args):
    sum = 0
    for number in args:
        sum += number
    return sum

print(f"This functions adds all the numbers passed to it and returns the sum =
{add(10,20,30)}\n")
```

Passing a variable number of arguments to functions

Used when not sure how many arguments will be passed

Syntax:

```
def func_name(*args):
    func_body
```

Args are received as tuple by the function

All tuple methods are available to process the args

Python Code: 2_fn_var_kwargs.py

```
#
#   Defining a function to receive a variable number of keyword arguments
#
def fn(**kwargs):
    print(f"This function receives a {type(kwargs)} consisting of the key:value
pairs: {kwargs}, which are {len(kwargs)} in number")

fn(greeting="Hello", second="World", number=1)
#
#   The function can use all dictionary methods
#
def groceries(**articles):
    print(f"The {len(articles)} items purchased are:")
    for article in articles.keys():
        print(article)
    total = 0
    for amount in articles.values():
        total += amount
    return total
#
#   Calling the function with a variable number of keyword args
#
print(f"and the value of items is: {groceries(bananas=5, tissues=6, oil=7)}\n")
print(f"and the value of items is: {groceries(bananas=5, tissues=6, oil=8,
soap=10)}\n")
print(f"and the value of items is: {groceries(bananas=12, tissues=7)}\n")
```

Passing a variable number of keyword arguments to functions

Used when not sure how many arguments will be passed

Syntax:

```
def func_name(**kwargs):
    func_body
```

Args are received as a dictionary by the function

All dictionary methods are available to process the args

Including: keys(), values(), items() etc.

Python Code: 3_fn_lambda.py

```
#
#   Simple lambda function useful for classifying data
#
fn_sort = lambda x: "Positive" if x>0 else "Zero" if x==0 else "Negative"
print(fn_sort(0))
print(fn_sort(10))
print(fn_sort(-10))

#
#   Using lambda functions with list comprehension for populating lists
#
```

```

fn2 = lambda x: x**2 + 3*x + 10
li = [fn2(x) for x in range(5)]
print(li)

#
# Using a lambda function to return >1 expression: Trick - return a tuple
#
fn3 = lambda a,b,c : (a**2+b**3, (a-b)**2, a*b*c)
my_tup = fn3(1,2,3)
print(my_tup)
#
# Unpacking the tuple
#
a, b, c = my_tup
print(a, b, c)

```

Lambda Functions

Lambda Functions are small anonymous functions

Take any number of arguments

Consist of a single expression only which is the return value

(Trick: use a tuple to return multiple expressions!)

Syntax:

lambda arguments : expression

where lambda = keyword

Lambda expression can include conditional statements

E.g a simple data classification function:

lambda x: "Positive" if x>0 else "Zero" if x==0 else "Negative"

Can be used in combination with other Python functions

Python Code: 4_fn_filter.py

```

def test_young(age):
    if (age <= 20):
        return True
    else:
        return False

def test_middle(age):
    if (age > 20 and age < 60):
        return True
    else:
        return False

def test_old(age):
    if (age >= 60):

```

```

        return True
    else:
        return False

#
#   Q: What are the ages of people classified as young, middle-aged or old?
#
my_list = [10, 20, 30, 40, 50, 60, 70, 80]
young_ones = filter(test_young, my_list)
middle_aged = filter(test_middle, my_list)
old_people = filter(test_old, my_list)
print("\nUsing three separate test functions with filter:")
print(f"Young people are aged: \t{list(young_ones)}\nMiddle aged are: \t{list(middle_aged)}\nand old folks are \t{list(old_people)}\n")

young_ones = filter(lambda x: True if x <= 20 else False, my_list)
middle_aged = filter(lambda x: True if 60 > x > 20 else False, my_list)
old_people = filter(lambda x: True if x >= 60 else False, my_list)
print("\nUsing lambda functions with filter:")
print(f"Young people are aged: \t{list(young_ones)}\nMiddle aged are: \t{list(middle_aged)}\nand old folks are \t{list(old_people)}\n")

```

Lambda Functions are small anonymous functions

Take any number of arguments

Consist of a single expression only which is the return value

(Trick: use a tuple to return multiple expressions!)

Syntax:

lambda arguments : expression

where lambda = keyword

Lambda expression can include conditional statements

E.g a simple data classification function:

lambda x: "Positive" if x>0 else "Zero" if x==0 else "Negative"

Can be used in combination with other Python functions

Other functions where lambda can be used

filter()

returns an iterator where the items are filtered through a function to test if the item is accepted (True) or not (False)

filter_obj = filter(test_func, iterable)

Test_func could be a lambda function

`map()`

executes a specified function for each item in an iterable

```
map_obj = map(func, iterable)
```

func could be a lambda function

`sorted()`

returns a sorted list of the specified iterable object

The sorting key may be provided as a function

Sorting can be ascending or descending

Default: strings are sorted alphabetically, numbers numerically

Syntax:

```
sorted_obj = sorted(iterable, key=key, reverse= reverse)
                (Key and reverse are optional)
```

Key could be a lambda function

Some other useful functions

`zip(iter1, iter2,...)`

Takes 2 or more iterators and returns a zip object with

One members from each iterator

The zip object can be iterated over

The length of the zip object is that of the shortest iterator passed as parameter

Generators

Used to create iterators, so that they can be used in for loops

Like functions but

Use yield statement to return a value

Return values on demand only

Do not run all the way but remain paused till next call

Huge memory and processing savings

Basic structure:

```
def function_name():
    some processing
    yield statement
```

May also be created through comprehension using `()`

Python Code: 5_fn_map.py

```
def map_func(x):
    return len(x)

my_list = ["apple", "banana", "cherry", "watermelon"]
print("\nUsing a function with map:")
print(f"The list is {my_list}")
print(f"and the number of characters in each list element are: {list(map(map_func, my_list))}")

print("Using a lambda function with map():")
print(f"The list is {my_list}")
print(f"and the number of characters in each list element are: {list(map(lambda x: len(x), my_list))}")
```

map()

executes a specified function for each item in an iterable

```
map_obj = map(func, iterable)
```

func could be a lambda function

Python Code: 6_fn_sorted.py

```
#
# Simple use of sorted() with iterable and default sorting
#
my_list = ["banana", "apple", "cherry", "watermelon"]
new_list = sorted(my_list)
print(f"The original list was:\n{my_list}\nand the sorted list is\n{new_list}\n")
#
# Simple use of sorted() with iterable and reverse sorting
#
new_list = sorted(my_list, reverse=True)
print(f"The original list was:\n{my_list}\nand the reverse sorted list is\n{new_list}\n")
#
# Use of sorted() with iterable and in-line key function
#
new_list = sorted(my_list, key=len)
print(f"The original list was:\n{my_list}\nand the sorted list based on length is\n{new_list}\n")

def sort_key2(x):
    return (x[0])

my_list = [(18, "Zulfiqar"), (50, "Abdul"), (25, "Babar")]
new_list = sorted(my_list, key=sort_key2)
print(f"The original list was:\n{my_list}\nand the list sorted by age using a
```

```

sort function is\n{new_list}\n")

new_list = sorted(my_list, key=lambda x: x[1])
print(f"The original list was:\n{my_list}\nand the list sorted by name using a
lambda function is\n{new_list}\n")

new_list = sorted(my_list, key=lambda x: x[1], reverse=True)
print(f"The original list was:\n{my_list}\nand the list reverse sorted by name
using a lambda function is\n{new_list}\n")

```

sorted()

returns a sorted list of the specified iterable object

The sorting key may be provided as a function

Sorting can be ascending or descending

Default: strings are sorted alphabetically, numbers numerically

Syntax:

```

sorted_obj = sorted(iterable, key=key, reverse= reverse)
                (Key and reverse are optional)

```

Key could be a lambda function

Python Code: 7_fn_zip.py

```

#
#   The zip function takes 2 or more iterators and returns a zip object which
is an iterator
#
list1 = [1,2,3,4,5]
tuple1 = ("Abdul", "Babar", "Chaudhry", "Daniyal", "Effendi", "Farooq")
dict1 = {1:"House", 2:"Apartment", 3:"Flat", 4:"Room"}

myzip = zip(list1, tuple1)
print(myzip)

for item in myzip:
    print(item)
print()
myzip = zip(list1, tuple1, dict1)
#
#   Uses key from each dictionary item for zip purposes
#
for item in myzip:
    print(item)

```

zip(iter1, iter2,...)

Takes 2 or more iterators and returns a zip object with

One members from each iterator

The zip object can be iterated over

The length of the zip object is that of the shortest iterator passed as parameter

Used to create iterators, so that they can be used in for loops

Like functions but

Use yield statement to return a value

Return values on demand only

Do not run all the way but remain paused till next call

Huge memory and processing savings

Basic structure:

```
def function_name():  
    some processing  
    yield statement
```

May also be created through comprehension using ()

Python Code: generator_0_intro.py

```
#  
# A simple gerator function  
# Uses the yield statement instead of return  
#  
def sum_upto(n):  
    sum = 0  
    i = 0  
    while i <= n:  
        yield sum  
        i += 1  
        sum += i  
  
#  
# Main program to test the generator  
#  
# Create a generator object  
#  
cumulative_sum = sum_upto(30)      # Maximum sum upto first 30 integers  
print(f"The object created is: {cumulative_sum}")  
count = 0                          # Loop counter  
#  
# Using a while loop requires next() to retrieve the value  
#  
print("First results using a while loop:")  
while count < 10:  
    print(f"The sum of the first {count} integers is: {next(cumulative_sum)}")  
    count += 1  
print("Another while loop. Next() continues from where it left off:")  
while count < 20:  
    print(f"The sum of the first {count} integers is: {next(cumulative_sum)}")  
    count += 1
```

```
#
# The for loop using the membership operator loops through the values yielded
by the generator
# The next() call is not required. Loop continues with the current state of
the generator
#
print("Now using the for loop with membership operator. Continues to the end of
the generator.")
for i in cumulative_sum:
    print(f"The sum of the first {count} integers is: {i}")
    count += 1
print("The generator has run to the end so the following loop never runs:")
for i in cumulative_sum:
    print(f"The sum of the first {count} integers is: {i}")
    count += 1
print("Program finished")
```